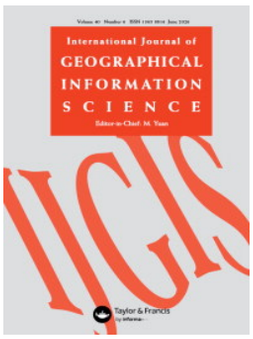

International Journal of Geographical Information Science

ISSN: 1365-8816 (Print) 1362-3087 (Online) Journal homepage: www.tandfonline.com/journals/tgis20

QQESPM-GS: an efficient solution for geo-textual search parameterized by graph-based spatial patterns

Carlos Vinicius Alves Minervino Pontes, Claudio E. C. Campelo, Maxwell Guimarães de Oliveira & Salatiel Dantas Silva

To cite this article: Carlos Vinicius Alves Minervino Pontes, Claudio E. C. Campelo, Maxwell Guimarães de Oliveira & Salatiel Dantas Silva (09 Feb 2026): QQESPM-GS: an efficient solution for geo-textual search parameterized by graph-based spatial patterns, International Journal of Geographical Information Science, DOI: 10.1080/13658816.2026.2620025

To link to this article: https://doi.org/10.1080/13658816.2026.2620025

INTERNATIONAL JOURNAL OF GEOGRAPHICAL INFORMATION SCIENCE
https://doi.org/10.1080/13658816.2026.2620025

Taylor & Francis
Taylor & Francis Group

RESEARCH ARTICLE

QQESPM-GS: an efficient solution for geo-textual search parameterized by graph-based spatial patterns

Carlos Vinicius Alves Minervino Pontes, Claudio E. C. Campelo, Maxwell Guimarães de Oliveira and Salatiel Dantas Silva

Systems and Computing Department, Federal University of Campina Grande, Bairro Universitário, Campina Grande, PB, Brazil

ABSTRACT

Efficient retrieval of geo-textual data is crucial in various applications. While existing research focuses on basic geo-textual queries, such as range and kNN queries, optimizing efficiency for complex queries remains a challenge. This study explores Quantitative and Qualitative Spatial Pattern Matching (QQ-SPM) queries, and showcase their applicability in Point of Interest (POI) search scenarios. This query format allows the search for groups of closely-located and relevant geo-textual objects based on a spatial pattern graph with keywords, pairwise distance limits, and topological constraints. We propose the Quantitative and Qualitative Efficient Spatial Pattern Matching Generalized Solution (QQESPM-GS), an approach designed to efficiently process such queries across different geospatial technologies. QQESPM-GS includes three primary solutions, each one designed to work with specific environments and spatial search engines: QQESPM-Quadtree, utilizing the IL-Quadtree index; QQESPM-EO, applying spatial operations with a greedy strategy; and QQESPM-SQL, enabling efficient execution within a relational spatial database. Experiments with a real POI dataset from UK, show that our approach remains robust with increasing dataset sizes and query constraints. Additionally, it achieves at least one order of magnitude faster query times compared to baseline methods for solving QQ-SPM queries.

ARTICLE HISTORY
Received 15 February 2025
Accepted 17 January 2026

KEYWORDS
Spatial-keyword query; geo-textual data search; spatial pattern matching; qualitative spatial reasoning; topological relationships

1. Introduction

The widespread adoption of technologies such as GPS, mobile internet and location-based services has led to a significant increase in the generation and consumption of geo-textual data daily (Zhang et al. 2016; Li et al. 2019; Chen et al. 2022, 2021). The rise of smartphones has driven a growing demand for keyword search systems and fostered the development of efficient indexing methods and algorithms to handle large volumes of geo-textual data effectively (Fogliaroni et al. 2016; Li et al. 2016b; Chen *et al.* 2020). A geo-textual object is characterized by a geographical location attribute and a textual attribute and may also include additional attributes. Examples

CONTACT Carlos Vinicius Alves Minervino Pontes carlosminervino@copin.ufcg.edu.br

of geo-textual objects include geo-tagged tweets, Points of Interest (POIs) data, check-in data, and any geo-referenced web content (Cao et al. 2015; Cong and Jensen 2019; Chen et al. 2021; Xu et al. 2022).

While there are numerous algorithms and indexing techniques for common geo-textual queries such as range and *k* Nearest Neighbor (kNN) queries, more complex queries involving diverse types of geo-textual objects have received relatively little attention. Specifically, efficient solutions for advanced geo-textual queries that incorporate complex spatial constraints among objects are limited (Guo et al. 2015; Deng et al. 2017; Fang et al. 2018a; Chen et al. 2022). The *m*-Closest Keyword (*m*CK) query is designed to retrieve groups of objects that collectively satisfy a set of *m* keywords while minimizing the distance among them. For example, an *m*CK query might identify a group of POIs, such as a school, a residential building, and a park, with the shortest possible maximum distance between them. Conversely, the Spatial Pattern Matching (SPM) query retrieves groups of objects based on keyword matching and pairwise distance constraints, such as locating a school, a residential building, and a park where the building is within specified distances from both.

Despite the tools available on geospatial search platforms, there is a lack of standard methods for representing queries based on diverse spatial pattern requirements, such as those in SPM queries. This absence can make complex queries frequently slow or suboptimal, highlighting the need for standardized guidelines for building efficient query plans and algorithms for complex spatial pattern searches with various constraints as parameters. Additionally, the computational cost of executing complex spatial pattern searches inefficiently can be time-consuming. Although indexing methods and algorithms for basic geo-textual queries have been well-explored (Cong et al. 2009; Wu et al. 2012; Zhang et al. 2016), few studies focus on optimizing performance for more complex spatial pattern searches. For example, current methods for *m*CK and SPM queries do not sufficiently represent search scenarios that involve both quantitative and qualitative constraints.

The following example illustrates a search requirement that cannot be represented by standard query types. Consider an individual looking to rent space in a commercial building for a small business (Minervino et al. 2023; Pontes et al. 2025). This person desires a location with an on-site gym, and, ideally, the commercial building should also be within a distance more than 2.5 and less than 5.5 km of an elementary school for their child's convenience. They may not want the building to be too close to the school for diverse reasons, like to avoid their child to potentially become confident to leave the school alone when it is dangerous. This scenario can be represented using a spatial pattern graph that incorporates both quantitative constraints (e.g. distance limits) and qualitative constraints (e.g. topological relationships), as depicted in Figure 1(A). The existence of algorithmic solutions optimally designed for the mentioned types of geo-textual requirements could enable the efficient execution of such a query. Other practical scenarios, quantitative and qualitative query requirements could include finding an apartment not too close to a cemetery or finding a building surrounded by a beautiful wooded green area.

In this study, we systematically explore the Quantitative and Qualitative Spatial Pattern Matching (QQ-SPM) query, a versatile geo-textual query that is more comprehensive than traditional spatial queries discussed in the literature. This query format allows for the incorporation of multiple keywords, pairwise distance constraints,

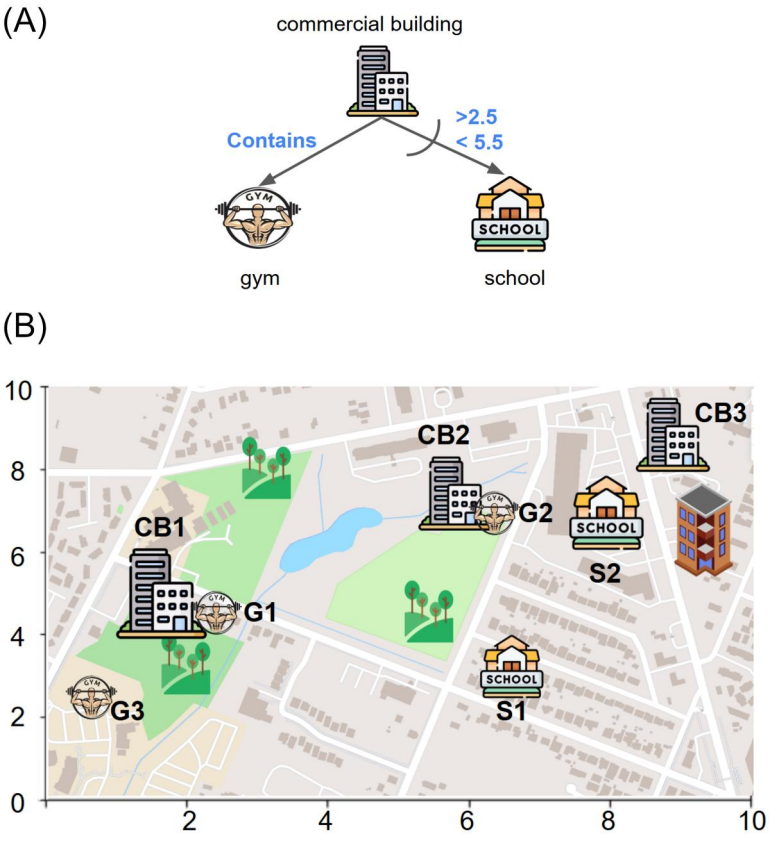

Figure 1. Map visualization of a fictitious dataset of POIs for the search example.

exclusion criteria, and topological relationships. To efficiently handle such queries, we introduce the Quantitative and Qualitative Efficient Spatial Pattern Matching Generalized Solution (QQESPM-GS), a robust approach designed to operate across various geospatial querying technologies. To our knowledge, there has been no systematic effort in the literature to efficiently address such a complex type of geo-textual query. As a result, there is a lack of specialized algorithms capable of answering queries with this type of configuration efficiently and effectively.

QQESPM-GS consists of three distinct solutions. Each solution represents a different resolution method for solving QQ-SPM queries, and each one is designed to work optimally for a specific environment or search engine: QQESPM-Quadtree, QQESPM-EO and QQESPM-SQL. QQESPM-Quadtree utilizes the Inverted Linear Quadtree (IL-Quadtree) indexing method; QQESPM-EO implements a greedy algorithm that applies a set of three elementary geo-textual operations (keyword search, radius search and topological search) to resolve QQ-SPM queries; QQESPM-SQL converts geo-textual requirements of a QQ-SPM query into an optimized SQL query that can be executed within a spatial relational database environment. The proposed QQESPM approach facilitates a systematic way of representing and efficiently answering geo-textual object retrieval tasks that involve graph-based spatial patterns, keywords, distances, and topological constraints.

QQ-SPM queries are beneficial in various practical geo-textual search applications, such as POI search following specific criteria. One such example arises in trip planning or the selection of suitable residential areas based on specific user keywords and spatial requirements. Beyond POI search, for instance, one might query for groups of geo-tagged social media posts that follow a particular spatial pattern using a QQ-SPM query. Additionally, the proposed solutions can serve as alternative query strategies for more generic spatial pattern searches that can be expressed in the QQ-SPM query format. Integrating these solutions into geospatial technologies, such as Google Maps and PostGIS, may enhance the performance of geo-textual queries that align with the QQ-SPM query format within these platforms.

In summary, our contributions are as follows:

- We developed the QQESPM-GS approach, which applies optimized algorithms to efficiently solve queries based on spatial pattern template graphs across multiple geospatial technologies, enhancing response time. An open-source implementation is also made available.
- We introduce the QQESPM-Quadtree algorithm as an adapted version from the QQESPM algorithm proposed by previous research (Minervino et al. 2023; Pontes et al. 2025). QQESPM-Quadtree utilizes the IL-Quadtree index, memoization techniques, and optimized join ordering to efficiently address QQ-SPM queries through a specialized approach.
- We propose the QQESPM-EO algorithm, which manages the efficient invocation of elementary spatial operations in a Geographic Information Retrieval (GIR) system to effectively solve a QQ-SPM query, and we showcase its applicability to enrich Elastisearch geospatial query functionality.
- We propose the QQESPM-SQL method, and a pipeline that translates geo-textual requirements from a QQ-SPM graph into a streamlined SQL spatial query that can be executed on a spatial relational database system. We showcase its applicability to enrich PostgreSQL/PostGIS geospatial query functionality
- We execute a performance evaluation to assess the efficiency of the three proposed solutions (QQESPM-Quadtree, QQESPM-EO, and QQESPM-SQL) against existing methods of solving the investigated QQ-SPM queries.

The remainder of the article is structured as follows: Section 2 outlines the fundamental concepts that underpin the design of the proposed solutions. Section 3 provides a review of related work on enhancing the efficiency of geo-textual queries. Section 4 introduces the proposed solutions: QQESPM-Quadtree, QQESPM-EO, and QQESPM-SQL. Section 5 offers a performance comparison for the proposed and for the existing methods for solving QQ-SPM queries. Finally, Section 6 concludes the article and suggests future directions for this research.

2. Background

This section reviews the essential foundational concepts for the formalization of the QQ-SPM problem and for the design of the algorithms proposed in the following section.

2.1. Inverted linear quadtree

The Inverted Linear Quadtree (IL-Quadtree) index (Zhang et al. 2016) is designed as a geo-textual indexing solution to enhance the processing of the top-k spatial keyword search problem, offering an improvement over existing indexing methods. This structure excels in handling data objects with both spatial attributes (location) and associated keywords. The IL-Quadtree indexing involves maintaining a separate Quadtree index for each unique keyword within the dataset. Consequently, each Quadtree indexes the objects related to its specific keyword.

The Quadtree (Finkel and Bentley 1974) is a spatial index that partitions a two-dimensional Euclidean space into four quadrants. The indexing begins with a universal bounding rectangle encompassing all spatial data points, which acts as the root node of the Quadtree. When the number of data points within a node exceeds a predetermined threshold, the node is divided into four child nodes using horizontal and vertical lines through the center. These child nodes represent the southwest, southeast, northwest, and northeast quadrants, encoded as binary numbers 00, 01, 10, and 11 (or decimal numbers 0, 1, 2, and 3), respectively. Figure 2(A) illustrates the space subdivision process of a Quadtree, while Figure 2(B) shows the resulting hierarchical tree structure. Each rectangular subdivision corresponds to a node in the tree. If any of these nodes at the first depth level exceeds the data point limit, it is further subdivided into four smaller nodes with identical bounding rectangles. This hierarchical subdivision continues recursively until all leaf nodes contain fewer objects than the threshold. For instance, in Figure 2, with a threshold of 1, the northwest node at the first depth level (N_2) containing two spatial objects (O_1 and O_2) is divided into four new nodes (N_{20}, N_{21}, N_{22}, and N_{23}).

As detailed by Zhang et al. (2016) and Chen *et al.* (2020), IL-Quadtree indexing has demonstrated efficiency in handling typical spatial keyword queries. The ESPM algorithm, introduced by Chen *et al.* (2020), leverages IL-Quadtree indexing to efficiently address Spatial Pattern Matching (SPM) queries. In Section 4, we present the QQESPM-Quadtree algorithm, which employs the IL-Quadtree indexing for solving QQ-SPM queries efficiently. The proposed method generalizes the classical SPM query discussed in previous research (Fang et al. 2018a; Chen *et al.* 2020).

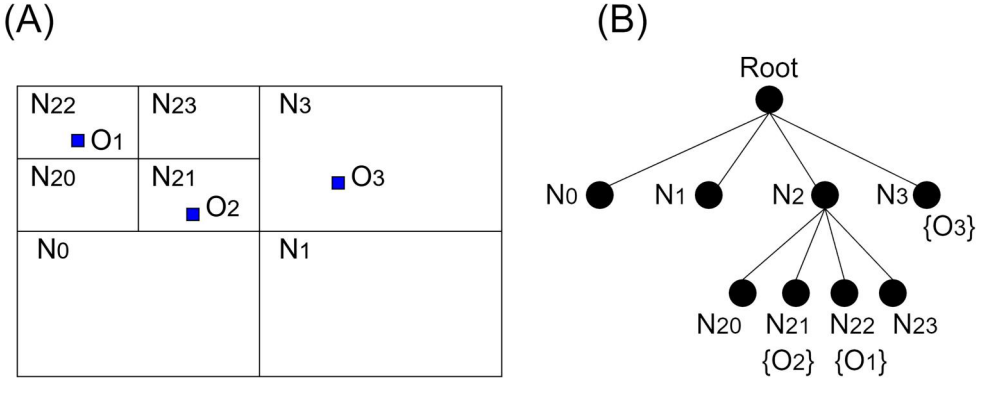

Figure 2. Example of a quadtree space subdivision (A) and its associated tree index structure (B). Adapted from Minervino et al. (2023).

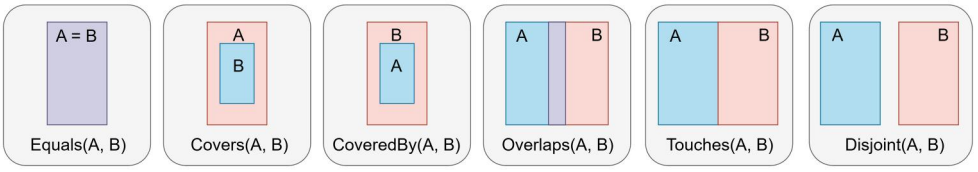

Figure 3. Examples of binary topological relationships.

2.2. Topological relationships

Qualitative spatial reasoning (QSR) involves developing calculi that enable machines to represent and reason about spatial entities while abstracting from metrical details (Freksa 1991; Cohn and Renz 2008; Moratz and Ragni 2008; Long et al. 2016). As outlined by Moratz and Ragni (2008) and Carniel (2023), QSR primarily addresses two main scenarios: topological relation reasoning (including spatial predicates such as 'touches', 'contains' and 'traverses') and orientation reasoning (which involves cardinal directions or other orientation-related aspects). This work concentrates on the topological reasoning aspect.

Topological relationships possess distinct properties: they are invariant to rotation, translation, and scaling. These relationships use qualitative spatial predicates to represent information about the connectivity or topological aspects between two spatial regions (or geometries) (Bedford 1993; Clementini and Di Felice 1995). This work focuses on binary topological relationships between two-dimensional spatial objects. Figure 3 provides examples of core binary topological relationships between spatial regions. The proposed solutions requires that qualitative query constraints have a formal definition and unambiguous computational representation. This necessitates using a formal topological model to encode the query constraints. Two widely recognized models for standardizing formal definitions of topological relationships are the Region Connection Calculus (RCC) (Randell et al. 1992; Cohn et al. 1997) and the Dimensionally Extended 9-Intersection Model (DE-9IM) (Egenhofer 1990; Egenhofer and Herring 1990; Clementini et al. 1993, 1994). Both models can be used in implementing the QQ-SPM search algorithms.

This research focuses on four topological relationships. In RCC, they are termed 'Connected', 'Disconnected', 'Part/Within' and 'Part Inverse/Contains'. These correspond to the DE-9IM relationships 'Intersects', 'Disjoint', 'CoveredBy' and 'Covers'. Hence, the qualitative topological constraints for QQ-SPM queries in this work are confined to these four predicates. This choice aims to simplify the formalization of QQ-SPM queries presented here and should not be seen as limiting the flexibility for future research on broader representations of QQ-SPM queries.

3. Related work

In this section, we discuss existing research on spatial and geo-textual queries, classifying geo-textual queries into nine categories based on their constraints. These categories are detailed as follows.

Traditional item-wise queries involve retrieving individual objects that meet specific geo-textual criteria. Examples include range queries (e.g. identifying all supermarkets

within 1 km of a given house) (Finkel and Bentley 1974; Guttman 1984; Christoforaki et al. 2011; Chen et al. 2013; Lee et al. 2015; Fevgas and Bozanis 2019; Roumelis et al. 2020). Another type is *k*-nearest neighbor (*k*NN) queries (e.g. locating the top five closest supermarkets to a specific house) (Cary et al. 2010; Chen et al. 2013; Lu et al. 2013; Tao and Sheng 2014; Yang et al. 2015; Zhang et al. 2016; Fevgas and Bozanis 2019). Additionally, top-*k* queries minimize a cost function that evaluates the relevance of objects based on keywords and location (De Felipe et al. 2008; Cong et al. 2009; Khodaei et al. 2010; Li et al. 2011; Rocha-Junior et al. 2011; Wu et al. 2012; Chen et al. 2013; Zhang et al. 2013; Mehta et al. 2016; Sun et al. 2017; Qian et al. 2018; Kalamatianos et al. 2021; Tampakis et al. 2021; Xu et al. 2022). Most spatial and geo-textual indexes and algorithms are designed to optimize these traditional item-wise queries.

Few studies address Item-wise queries with distance and directional constraints (Li et al. 2012; Guo and Yang 2019). These include range, *k*NN, and top-*k* queries that incorporate spatial directional reasoning along with distance metrics.

Topological item-wise queries, such as point and region queries (Fevgas and Bozanis 2019; Carniel et al. 2020), identify spatial objects with certain topological relationships to a fixed point or region.

Despite their important application scenarios, item-wise queries are not as powerful as the QQ-SQPM queries investigated in this research, since they do not allow retrieving groups of related, closely located objects at once. While these queries may even incorporate qualitative topological constraints, they are either purely spatial lacking keyword search capabilities, or are item-wise, only retrieving single objects.

In contrast to item-wise queries, Group Queries (Chen *et al.* 2020, Chen et al. 2021) are designed to retrieve groups of objects where each result consists of a collection of related items. These queries are further classified and discussed.

Collective Spatial Keyword Queries (CoSQKs) are group-oriented geo-textual queries that retrieve the groups of spatial objects meeting the query's keywords while minimizing a specific cost function that assesses the relevance of these object groups to the query's keywords and location. A classic example is the *m*-Closest Keyword (*m*CK) query, which identifies groups with the smallest diameters, defined as the maximum distance between any two objects in the group (Zhang et al. 2009; 2010; Guo et al. 2015). Various studies utilize different cost functions to simultaneously minimize inter-object distances, the distance to the query's central location, and other factors to create a comprehensive relevance metric (Long et al. 2013; Zhang et al. 2014, 2017; Cao et al. 2015, 2011; Deng et al. 2015; Skovsgaard and Jensen 2015; Choi et al. 2016; Chan et al. 2017; 2018; Hermoso et al. 2019; Choi et al. 2020). In comparison with the flexibility of QQ-SPM queries addressed in our research, a key limitation of these queries is their inability to represent pair-wise spatial constraints between specific objects in a group, as these queries generally focus on minimizing a single group-wide metric.

We employ the term Neighborhood Optimization Queries (NOQs) to describe geo-textual queries aimed at identifying optimal spatial objects based on a score that evaluates the relevance of surrounding objects to the query's keywords. Essentially, an NOQ involves a primary keyword denoting the main searched object (referred to as

the data object) and a set of keywords representing user preferences for nearby objects (known as feature objects). Several studies (Yiu et al. 2007; de Almeida and Rocha-Junior 2015; Tsatsanifos and Vlachou 2015; Gao et al. 2016; Li et al. 2016a; Deng et al. 2017) have proposed and refined slight variations of this query format. We further categorize these studies into three types: NOQs without distance constraints (Li et al. 2016a); traditional NOQs with distance constraints (Yiu et al. 2007; de Almeida and Rocha-Junior 2015; Tsatsanifos and Vlachou 2015; Gao et al. 2016); and NOQs with both distance and directional constraints (Deng et al. 2017). A key limitation of NOQs, in comparison with QQ-SPM queries, is that they focus on the central searched data object and do not retrieve the exact locations of the relevant feature objects nearby.

The Spatial Pattern Matching (SPM) query retrieves groups of objects that conform to a specific spatial pattern, defined by a graph where vertices represent keywords for the target objects and edges denote distance constraints between spatial objects that match the vertices' keywords (Fang et al. 2018a; 2018b, Fevgas and Bozanis 2019; Li et al. 2019; Chen *et al.* 2020). Some studies propose slight modifications to the definition of spatial patterns in queries (Guo et al. 2020; Chen et al. 2022). However, a lack of flexibility of SPM queries and its traditional variations studied in the literature, in comparison of the QQ-SPM queries addressed in our research, is the lack of algorithms capable of handling spatial patterns with both quantitative and qualitative spatial constraints simultaneously.

The Qualitative Spatial Pattern (QSP) query (Rafael et al. 2024) is a group-oriented qualitative topological query that incorporates spatial patterns with both keywords and topological constraints for the target objects. However, unlike QQ-SPM, the QSP query does not accommodate customized pair-wise distance restrictions or exclusion constraints, for example, these query type cannot represent the following search scenario: finding residential buildings located at least 500 meters away from cemeteries.

Table 1 summarizes and compares related works by highlighting the features present and absent in these categories of geo-textual queries explored in the literature. The features examined include whether the:

Table 1. Present and absent features in categories of geo-textual queries in the literature and in the QQ-SPM query.

Query types	Geo-textual	Quantitative	Qualitative	Topological	Group search	Pair-wise constraints	Exclusion constraints
Traditional item-wise queries	√	√					
Item-wise queries with distance and directional constraints	√	√	√				
Topological item-wise queries			√	√			
CoSKQs	√	√			√		
NOQs with no distance limits	√	√			√		
Traditional NOQs with distance limits	√	√			√	√	
NOQs with distance limits and directional constraints	√	√	√		√	√	
SPM and its traditional variations	√	√			√	√	√
QSP	√		√	√	√	√	
QQ-SPM (This research)	√	√	√	√	√	√	√

- queries are geo-textual or purely spatial;
- queries incorporate quantitative constraints (e.g. distance range limits);
- queries include qualitative constraints (e.g. directional or topological restrictions);
- queries support topological constraints (e.g. contains, intersects);
- queries retrieve individual items or groups of items;
- queries accommodate pair-wise constraints (e.g. specifying spatial restrictions between only two specific objects);
- queries allow exclusion constraints (e.g. 'retrieve only POIs not located near cemeteries').

The QQ-SPM query can address both quantitative (distance) and qualitative (topological) requirements. It retrieves groups of spatial objects that collectively conform to a specified spatial pattern graph. This query type supports pair-wise spatial constraints between the objects and enables exclusion constraints, such as identifying POIs that must not be too close to certain types of facilities. QQ-SPM queries offer a versatile and flexible approach, allowing for queries defined by a complete spatial pattern graph with both quantitative and qualitative restrictions. The ability to represent complex search requirements differentiates QQ-SPM flexibility over other geo-textual query types. This query type has been initially investigated by Minervino et al. (2023) and Pontes et al. (2025), whose works proposed the QQESPM algorithm as a solution to such queries. This algorithm is a baseline on top of which we propose novel approaches.

4. QQESPM-GS

This section outlines the algorithms and pipelines that constitute QQESPM-GS, a comprehensive solution for geo-textual queries characterized by graph-based spatial patterns with both distance and topological constraints (the QQ-SPM queries). Section 4.1 reviews the formal definition of the QQ-SPM search problem. Section 4.3 describes the proposed QQESPM-Quadtree algorithm, which is inspired on previously existing QQESPM (Minervino et al. 2023; Pontes et al. 2025) and ESPM (Chen *et al.* 2020) algorithms. While ESPM is limited to solving queries based solely on spatial patterns with distance constraints, our proposed solution QQESPM-Quadtree efficiently addresses queries involving both distance and topological constraints. Section 4.4 explains the QQESPM-EO algorithm, an alternative method for solving QQ-SPM queries based on elementary spatial operations. Section 4.5 introduces QQESPM-SQL, a pipeline designed to automatically transform the spatial pattern graph of a QQ-SPM query into an efficient spatial SQL query. Figure 4 shows the architecture of a system equipped with the proposed QQESPM-GS framework for solving QQ-SPM queries. The three proposed solutions QQESPM-Quadtree, QQESPM-EO and QQESPM-SQL serve as different query backends, each aligned with a specific running environment or spatial search engine, which can independently run QQ-SPM queries efficiently. In our experiments, we showcase how to employ these solutions in three spatial search engines or environments: PostGIS, Elasticsearch, and a server with a fully adhoc IL-Quadtree python implementation, and open-source implementation is also provided.

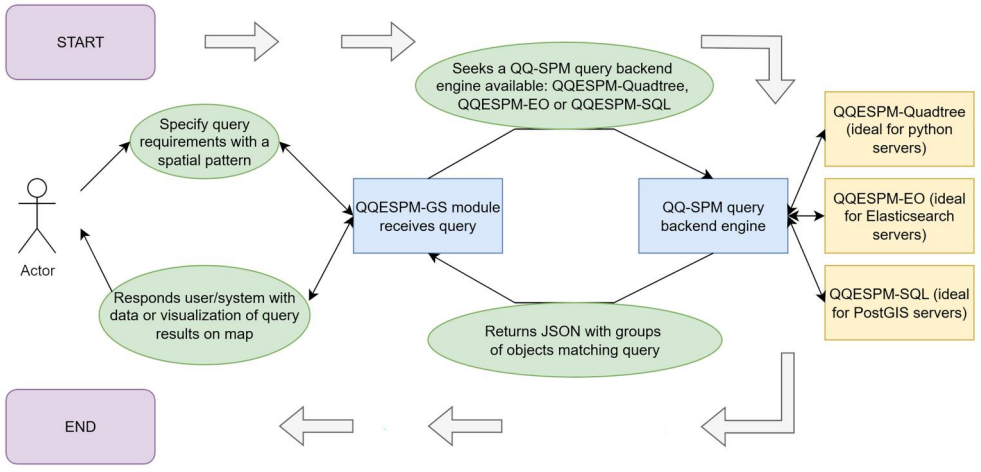

Figure 4. Architecture of a search system equipped with QQESPM-GS framework for QQ-SPM queries.

4.1. QQ-SPM problem definition

This section provides formal definitions of the key terms relevant to QQ-SPM queries. It establishes a clear delineation of QQ-SPM geo-textual queries as the primary subject of study. A spatial pattern is defined as a graph labeled with keywords and spatial constraints, which is used as a parameter in a query to identify groups of geo-textual objects that meet the spatial configuration or arrangement specified in the graph. The query is performed on a dataset D consisting of geo-textual objects. Each geo-textual object o includes a set of associated keywords, denoted as $o.doc$, and a spatial location, represented either as a point or a polygonal shape, denoted as $o.loc$. In the following, we give the formal definition of spatial pattern and match (adapted from Fang et al. 2018a; Chen *et al.* 2020, and following the standard from Minervino et al. 2023 and Pontes et al. 2025).

Definition 4.1. Spatial pattern (Minervino et al. 2023; Pontes et al. 2025). *A spatial pattern is a graph $G(V, E)$ represented by a set of vertices $V = v_1, \ldots, v_n$ and a set E of edges $e_{ij} = e(v_i, v_j)$, such that:*

1. *each vertex $v_i \in V$ has an associated keyword w_i*
2. *each edge $e_{ij} \in E$ stores the spatial constraints for a pair of objects matching the keywords of its endpoint vertices v_i and v_j. The possible constraints consist of:*
   - *a topological spatial predicate $\Re_{ij}$, which can be one among: $\{contains, within, intersects, disjoint\}$*
   - *a distance lower bound l_{ij}, and a distance upper bound u_{ij}*
   - *an exclusion sign $\tau \in \{\leftarrow, \rightarrow, \leftrightarrow, -\}$*

Each possible spatial pattern graph serves as a parameter for a QQ-SPM query. Taking as example a POI search scenario, the vertices could represent the desired keywords of the POIs, while the topological predicates specify the preferred relationships

between the POIs. As per Definition 4.1, the spatial pattern graph requires that for each vertex v_i, a corresponding geo-textual object o_i from the dataset must be identified. The keyword associated with vertex v_i must match one of the keywords in $o_i.doc$. Additionally, the distances and topological relationships between objects must conform to the spatial constraints defined by the graph's edges. The distances between the POIs are constrained by the lower (l_{ij}) and upper (u_{ij}) bounds associated with each edge. The meanings of the potential exclusion symbols for an edge are explained below:

- $v_i \rightarrow v_j$ [v_i excludes v_j]: There must be no geo-textual object with keyword w_j whose spatial location is within a distance shorter than l_{ij} from the geo-textual object matching v_i.
- $v_i \leftarrow v_j$ [v_j excludes v_i]: There must be no geo-textual object with keyword w_i whose spatial location is within a distance shorter than l_{ij} from the geo-textual object matching v_j.
- $v_i \leftrightarrow v_j$ [mutual exclusion]: Specifies that both constraints are applied: v_i excludes v_j and v_j excludes v_i.
- $v_i - v_j$ [mutual inclusion]: Geo-textual objects with keywords w_i and w_j are permitted to be within a distance shorter than l_{ij} from objects corresponding to v_i and v_j.

Edges with distance interval restrictions for the searched objects are called quantitative edges, while those with topological restrictions are referred to as qualitative edges. An edge can be both quantitative and qualitative at the same time. A quantitative edge that features one of the exclusion signs '$\leftarrow$', '$\rightarrow$', or '$\leftrightarrow$' is termed an exclusive edge, and the search constraint it represents is known as an exclusion constraint. Edges marked with the mutual inclusion sign ('$-$') are classified as inclusive edges.

For instance, consider an edge in the search pattern that connects vertices labeled with the keywords 'apartment' and 'school', with $l_{ij} = 100m$, $u_{ij} = 1000m$, and a mutual inclusion sign $\tau =$ '$-$'. An apartment A_1 located 105 m from a school S_1 will be included in the search results, regardless of whether other schools are within 100 m of A_1. However, if the edge is marked with the sign $\tau =$ '$\rightarrow$' between 'apartment' and 'school', for the pair of objects (A_1, S_1) to satisfy the edge conditions, there must also be no other school S closer than $l_{ij} = 100m$ to A_1. If such a school exists, the pair (A_1, S_1) will not appear in the search results, as the search pattern requires the apartment to be sufficiently distant from schools (exclusion constraint).

Definition 4.2. Matching object pair (adapted from Minervino et al. 2023; Pontes et al. 2025). *A pair of geo-textual objects (o_i, o_j) is defined as a matching pair for the edge $e(v_i, v_j)$ if o_i and o_j contain the keywords of the vertices v_i and v_j, respectively, and their spatial locations meet the constraints specified by the edge e.*

Definition 4.3. Match (Minervino et al. 2023; Pontes et al. 2025). *A tuple of n geo-textual objects $S = (o_1, o_2, \ldots, o_n)$ is defined as a match for a spatial pattern $G(V, E)$ if $|V| = n$, and for each i, the object o_i possesses the keyword associated with the vertex v_i. Additionally, for every edge $e(v_i, v_j)$ in G, the pair of objects (o_i, o_j) must be a matching pair for the edge e_{ij}.*

Note that the order of geo-textual objects in the tuple corresponds directly to the order of vertices in the pattern G. Thus, the i^{th} object o_i in the matching tuple aligns with the i^{th} vertex (v_i) in the pattern G. The QQ-SPM query format generalizes the SPM query by incorporating both SPM distance search criteria and additional topological constraints for defining spatial patterns.

Problem 4.4. QQ-SPM search problem (Minervino et al. 2023; Pontes et al. 2025). *The QQ-SPM search problem, or QQ-SPM query, involves identifying all matches of a spatial pattern $G(V, E)$ within a dataset D of geo-textual objects. In other words, it requires finding all groups of objects from D whose keywords and locations satisfy the conditions specified by the spatial pattern graph.*

4.2. QQ-SPM search example

The spatial pattern graph in Figure 1(A) specifies a query for a commercial building that must contain a gym and be situated between 2.5 and 5.5 distance units from a school. The edge 'commercial building – school' is marked as exclusive with the sign '$\rightarrow$', indicating that the resulting commercial buildings must not be located closer than 2.5 distance units to any other school. This exclusion constraint ensures that the selected commercial building is sufficiently distant from other schools. Let us denote the vertices of the search pattern (commercial building, gym, and school) as V_1, V_2, and V_3, respectively, and the edges 'commercial building – gym' and 'commercial building – school' as e_{CG} and e_{CS}, respectively.

The area of POIs illustrated in Figure 1(B) represents the target dataset for the query within a hypothetical 2-D Euclidean space. The POIs $CB1$, $CB2$, and $CB3$ are tagged with the keyword 'commercial building'; $G1$, $G2$, and $G3$ are associated with the keyword 'gym'; and $S1$ and $S2$ are labeled with the keyword 'school'. Here, the pair of POIs $(CB1, S1)$ forms a matching pair of objects (Def. 4.2) for the edge e_{CS} ('commercial building – school') because this pair satisfies the distance and exclusion constraints of the edge. Conversely, the pair $(CB2, S1)$ does not qualify as a matching pair for e_{CS} because the commercial building $CB2$ is too close to the school $S1$, violating the edge's exclusion constraint. Additionally, the pair $(CB1, G1)$ meets the criteria for the edge e_{CG}. Thus, the tuple $(CB1, G1, S1)$ constitutes a match (Def. 4.3) for the spatial pattern in the query (Figure 1(A)), as it includes one object for each vertex of the search pattern and fulfils the keywords, distance, and topological constraints.

4.3. QQESPM-quadtree

The proposed QQESPM-Quadtrees algorithm is based on two theorems established in this section. The core idea of the algorithm involves filtering promising nodes and objects for the edges, followed by a final join phase to complete the solutions for the entire search graph. The QQESPM-Quadtree algorithm requires the geo-textual object dataset to be indexed in an IL-Quadtree structure, with each keyword having its own separate quadtree containing all objects associated with that keyword. A solution for an edge comprises a pair of objects that match the keywords at the edge's endpoints

and meet the edge's spatial constraints. Accordingly, a matching pair of objects for an edge $e(v_i, v_j)$ must be sought within a pair of nodes (N_i, N_j) from the quadtrees of the keywords for vertices v_i and v_j, respectively. The QQESPM-Quadtree algorithm first filters promising pairs of nodes for each edge. These are node pairs that potentially contain objects forming a matching object pair for that edge.

4.3.1. Base theorems for QQESPM-quadtree

Next, we define a set of conditions to characterize a promising node pair for an edge. We then demonstrate that this definition effectively identifies all matching object pairs for an edge, proving that they are necessarily within these so-called promising node pairs.

Definition 4.5. Promising pair of nodes (adapted from Minervino et al. 2023; Pontes et al. 2025). *Let ILQ denote an IL-Quadtree of geo-textual objects, $G(V, E)$ represent a spatial pattern graph, and $e(v_i, v_j)$ be an edge of G. Let ILQ_i and ILQ_j be the quadtrees for the keywords of vertices v_i and v_j, respectively, with n_i and n_j as nodes from ILQ_i and ILQ_j, respectively, and b_i and b_j as their bounding boxes. The node pair (n_i, n_j) is termed a promising pair for the edge $e(v_i, v_j)$ if $d_{\min}(b_i, b_j) \leq u_{ij}$ (distance upper bound between v_i and v_j), $d_{\max}(b_i, b_j) \geq l_{ij}$ (distance lower bound between v_i and v_j), and additionally, if the following conditions are met:*

- *Case $v_i \rightarrow v_j$: There is no object x_j in ILQ_j such that $d_{\max}(b_i, x_j) < l_{ij}$.*
- *Case $v_i \leftarrow v_j$: There is no object x_i in ILQ_i such that $d_{\max}(x_i, b_j) < l_{ij}$.*
- *Case $v_i \leftrightarrow v_j$: There is no object x_j in ILQ_j with $d_{\max}(b_i, x_j) < l_{ij}$ and no object x_i in ILQ_i with $d_{\max}(x_i, b_j) < l_{ij}$.*
- *Case e is qualitative with $\Re_{ij} \neq$ "disjoint" : $b_i \cap b_j \neq \emptyset$.*

The functions $d_{\min}$ and $d_{\max}$ denote the minimum and maximum distances between a bounding boxes and the centroid of a spatial object. According to this definition, edges with different signs impose different conditions for a node pair to be deemed promising. The concept of a promising pair of nodes is based on the following intuition: a pair of nodes (n_i, n_j) is considered promising for the edge e if the bounding boxes of these nodes potentially encompass matching pairs of objects for the edge e. Specifically, if the minimum and maximum distances between the bounding boxes b_i and b_j do not preclude the possibility of geo-textual objects o_i and o_j within these nodes forming a matching pair for the edge e, then the inner nodes or objects of n_i and n_j should be further analyzed. These node pairs are potential candidates for finding matching object pairs for the edge e.

Theorem 4.6. Adapted from Pontes et al. 2025). *Let $e(v_i, v_j)$ be an edge in a spatial pattern graph $G(V, E)$, and let ILQ_i and ILQ_j be the quadtrees for the keywords associated with vertices v_i and v_j, respectively. Consider a pair of geo-textual objects (o_i, o_j) in the dataset D. If (o_i, o_j) forms a matching pair for the edge e, then for any nodes n_i and n_j from the quadtrees ILQ_i and ILQ_j, respectively, if o_i is within n_i and o_j is within n_j, then the node pair (n_i, n_j) is a promising pair of nodes for e.*

Proof. Since (o_i, o_j) is a matching pair, it satisfies the constraints of the edge e. Specifically, we have $d_{\max}(b_i, b_j) \geq d(o_i, o_j) \geq l_{ij}$, and $d_{\min}(b_i, b_j) \leq d(o_i, o_j) \leq u_{ij}$. Additionally, if e is qualitative with $\Re_{ij} \neq$ "disjoint", then o_i and o_j intersect, implying that n_i and n_j also intersect. Now, consider the possible signs of the edge e. For the case $v_i \rightarrow v_j$, assume there is an object x_j in ILQ_j such that $d_{\max}(b_i, x_j) < l_{ij}$. Since o_i is within the bounding box b_i of node n_i, we have $l_{ij} > d_{\max}(b_i, x_j) \geq d(o_i, x_j)$, which contradicts the assumption that (o_i, o_j) is a matching pair. Hence, no such x_j exists. The proofs for other edge signs follow similarly, ensuring that all conditions for (n_i, n_j) to be a promising pair of nodes are satisfied. ■

Using Theorem 4.6 in a QQ-SPM search procedure, only promising pairs of nodes need interior verification to identify all matching objects. Objects o_i and o_j that are not within any promising pairs of nodes for an edge e cannot form a matching pair for e. In other words, no solutions for an edge e exist outside its promising pairs of nodes. Consequently, by concentrating solely on these promising pairs, an algorithm can locate all spatial pattern matches without examining the entire dataset. The following theorem presents the final component necessary for designing an efficient search for promising node pairs level by level.

Theorem 4.7. Adapted from Minervino et al. (2023), and Pontes et al. (2025). *If (n_i, n_j) is not a promising node pair for the edge $e(v_i, v_j)$, then any pair (n_i^c, n_j^c), where n_i^c is a child of n_i and n_j^c is a child of n_j, cannot be a promising pair of nodes for the edge $e(v_i, v_j)$.*

Proof. The proof for the distance constraints of the edges is provided in Chen *et al.* (2020). To address the additional constraint related to the topological requirement of the edge, we use the contrapositive of the statement. Specifically, if the node pair (n_i, n_j) is promising for an edge with a qualitative constraint other than 'disjoint', then their bounding boxes must intersect. Since these bounding boxes are covered by their parent nodes, the parent nodes themselves will also intersect. Therefore, if a pair of nodes is promising for an edge e, then their parent nodes must also form a promising pair for that edge. ■

By applying Theorem 4.7 in a QQ-SPM search procedure, the promising node pairs for the next level can be identified without comparing the bounding boxes of every node pair at the current depth of the quadtrees. Instead, the search needs only to examine node pairs that are children of the promising pairs from the previous level. This is because a node pair must be promising at the current level if its parent node pair is promising at the preceding level (Theorem 4.7). Utilizing this approach, the QQESPM-Quadtree search procedure efficiently identifies promising node pairs for each edge, level by level, by verifying only the children of the promising node pairs from the previous level down to the leaf level. Finally, the procedure can then verify the inner objects of the promising node pairs at the leaf level to find the matching objects.

4.3.2. QQESPM-Quadtree algorithm description

The QQESPM-Quadtree algorithm (Algorithm 4.3.2) follows a similar architecture to ESPM (Chen *et al.* 2020) and QQESPM (Minervino et al. 2023; Pontes et al. 2025) algorithms, computing promising node pairs for all edges level by level, finding matching

object pairs at the leaf level, and then joining these pairs to identify the groups that match the entire search pattern graph. However, there are several key differences between the two algorithms. First, ESPM filters promising node pairs based solely on distance constraints and similarly filters matching object pairs according to these distance restrictions. It does not address topological requirements, which must be verified later, after retrieving all object groups filtered only by distance constraints. In contrast, QQESPM-Quadtree filters both promising nodes and objects simultaneously by considering both distance and topological requirements. For example, for an edge with a non-disjoint topological constraint, promising node pairs are required to intersect, and promising object pairs are pre-filtered based on the intersections of the bounding boxes of the candidate objects. Thus, QQESPM-Quadtree takes into account both distance and topological restrictions throughout the entire search procedure level by level. Another significant difference is in the strategy for reordering edges during the final join phase. QQESPM-Quadtree employs a more efficient reordering strategy, which reduces the cost of the final join phase. This reordering strategy for the join phase is explained in detail in this section.

QQESPM-Quadtree takes as input a spatial pattern represented as a graph $G(V, E)$ and a dataset of geo-textual objects indexed in an IL-Quadtree *ILQ*, which includes quadtrees ILQ_i for each keyword w_i in the dataset. The algorithm aims to identify all matches of the spatial graph G within the geo-textual dataset.

Algorithm 1. QQESPM-Quadtree.

Input: IL-Quadtree *ILQ*, spatial pattern $G(V, E)$
Output: ψ: all the matches of G

$L = \max(depth(ILQ_i), 1 \leq i \leq n)$
foreach *edge* $e(v_i, v_j) \in E$ **do**
  $\Psi_{e(0)} \leftarrow \{(ILQ_i.\text{root}, ILQ_j.\text{root})\}$
foreach *each level* $l = 1$ *to* L **do**
  foreach *each edge* e **do**
    $\Psi_{e(l)} \leftarrow$ RUN Compute_Promising_Node_Pairs_Level(l, $\Psi_{e(l-1)}$, $G(V, E)$)
$E \leftarrow$ RUN Get_Greedy_Connected_Edges_Order(Ψ, $G(V, E)$, false)
$SE \leftarrow$ the skip-edges in sequence E according to Def. 4.9
foreach *each non-skip edge* $e \in E \backslash SE$ **do**
  $\Phi e \leftarrow$ the promising object pairs for e within the promising node pairs $\Psi_{e(l)}$ // filter based on distances or bounding box intersection
$E \leftarrow$ RUN Get_Greedy_Connected_Edges_Order(Φ, $G(V, E)$, true)// reorder the edges using a greedy alternating strategy
$\psi \leftarrow \emptyset$ // keep the partially constructed matches of the search pattern
foreach *each non-skip edge* $e \in E$ **do**
  $\psi \leftarrow \psi$.join(Φe)
foreach *each skip-edge* $e \in SE$ **do**
  filter out partial solutions in ψ not satisfying constraints of e
foreach *each edge* $e \in E$ *with topological constraint* **do**
  filter out partial solutions in ψ not satisfying the exact topological constraint of e
return ψ

The maximum depth level at which QQESPM-Quadtree operates corresponds to the deepest quadtree associated with the keywords in the search pattern (line 1 of Algorithm 1). For each edge, the initial promising node pairs are composed of the root nodes of the quadtrees for the keywords of the edge's endpoint vertices (lines 2–3 of Algorithm 1). The search procedure computes promising node pairs at each depth level of the quadtrees, utilizing the promising pairs from previous levels (lines 4–6 of Algorithm 1). The subroutine `Compute_Promising_Node_Pairs_Level`, invoked in line 6 of Algorithm 1, is detailed in Algorithm 2. This procedure takes the current depth level l and the promising node pairs from the previous level for each edge e, stored in the variable $\Psi_{e(l-1)}$, and computes the promising node pairs for each edge at the next depth level of the quadtrees.

Algorithm 2. Compute_Promising_Node_Pairs_Level.

Input: l: current quadtree depth level, Ψ: promising node pairs for edges from previous depth level, $G(V, E)$: spatial pattern
Output: Ψ: the promising node pairs up to depth level l of the quadtrees for each edge

1 $L = \max(depth(ILQ_i), 1 \leq i \leq n)$
2 $EE \leftarrow \{e \in E$: e is exclusive$\}$
3 $IE \leftarrow \{e \in E$: e is inclusive$\}$
4 Reorder EE in form $\{e_1, \ldots, e_k\}$ s.t. $\#\Psi_{e_1(l-1)} \leq \ldots \leq \#\Psi_{e_k(l-1)}$
5 Reorder IE in form $\{e_{k+1}, \ldots, e_m\}$ s.t. $\#\Psi_{e_{k+1(l-1)}} \leq \ldots \leq \#\Psi_{e_m(l-1)}$
6 $E \leftarrow$ the concatenation of the arrays EE and IE
7 **for** $v_i \in V$ **do**
8 $C_{i(l)} \leftarrow$ all nodes of ILQ_i at level l
9 **for** $e \in E$ **do**
10 $\Psi_{e(l)} \leftarrow \emptyset$
11 **for** $(n_i, n_j) \in \Psi_{e(l-1)}$ **do**
12 **if** $n_i \in C_{i(l)}$ $n_j \in C_{j(l)}$ **then**
13 **for** $(n'_i, n'_j) \in n_i.children \times n_j.children$ **do**
14 **if** (n'_i, n'_j) *is promising for edge e according to Def. 4.5* **then**
15 $\Psi_{e(l)}.add((n'_i, n'_j))$
16 $v_i, v_j \leftarrow$ the endpoint vertices of e
17 $N_i \leftarrow$ all nodes of ILQ_i at level l figuring at some promising pair of $\Psi_{e(l)}$
18 $N_j \leftarrow$ all nodes of ILQ_j at level l figuring at some promising pair of $\Psi_{e(l)}$
19 $C_{i(l)} \leftarrow C_{i(l)} \cap N_i$
20 $C_{j(l)} \leftarrow C_{j(l)} \cap N_j$
21 **return** Ψ

QQESPM-Quadtree prioritizes edges with signs '←', '→', or '↔' (exclusive edges) over those with sign '–' (inclusive edges) by reordering the edge list (lines 2–6 of Algorithm 2). This strategy optimizes the algorithm by addressing the most restrictive spatial constraints first, thus minimizing the number of candidate solutions considered during execution.

During the computation of promising node pairs for edges at a specific depth level, QQESPM-Quadtree maintains temporary sets of candidate nodes for each vertex in the search pattern (lines 7–8 of Algorithm 2). At each iteration for a given level l, the algorithm determines promising node pairs for the next level by analyzing the children of the promising node pairs from the previous level, as detailed in Theorem 2. However, the promising node pairs are restricted to those whose nodes are among the gathered candidate nodes associated with adjacent edges previously processed (lines 11–12 of Algorithm 2). For instance, for edges e_{12} and e_{13} sharing vertex v_1, the node for keyword w_1 in the promising node pairs of e_{13} must be among the nodes for w_1 in the promising pairs of e_{12}. This ensures that candidate solutions for subsequent edges are constrained by those processed earlier for adjacent edges, highlighting the need to prioritize more restrictive edges to filter nodes more effectively. After determining the promising node pairs for an edge, the candidate nodes for their endpoint vertices are updated (lines 17–20 of Algorithm 2). Consequently, after computing promising node pairs for each edge at a specific quadtree level, QQESPM-Quadtree reorders the edge list for the next level, prioritizing edges with fewer promising node pairs from the previous level.

After computing the promising node pairs for all edges at the leaf level, the edges are reordered into a new sequence $\{e_1, e_2, \ldots, e_m\}$, representing a connected order of edges generated using a greedy strategy (line 7 of Algorithm 1). Definition 4.8 specifies the criteria for an edge ordering to be considered connected. The reordering process is managed by the subroutine `Get_Greedy_Connected_Edges_Order`, which is detailed in Algorithm 3.

Definition 4.8. Connected Edges Order. *An order of edges $\Pi = (e_1, e_2, \ldots, e_m)$ in a spatial pattern graph G is considered a connected edges order if, for each edge e_k in Π, at least one of the endpoint vertices v_i or v_j of e_k is also an endpoint of at least one edge in the sub-sequence $(e_1, e_2, \ldots, e_{k-1})$ that precedes e_k.*

Algorithm 3: Get_Greedy_Connected_Edges_Order

Input: Ψ: promising node or object pairs for the edges, $G(V, E)$: spatial pattern, *alt*: a Boolean parameter (false specifies a greedy-minimum reordering, true specifies a greedy-alternating reordering

Output: Π: the edges from E in a greedy and connected reordering

```
1  Π ← ∅
2  e ← the edge from E that has the minimum #Ψ_e
3  Π.add(e)
4  B ← ∅
5  v_i, v_j ← the endpoint vertices of e
6  if v_i is incident to an edge besides e then
7  |  B.add(v_i)
8  v ← v_j //keep the current vertex
9  while E\Π ≠ ∅ (there are edges left) do
10 |  N ← {v' ∈ v.neighbors: e(v,v')∉Π}
11 |  if N ≠ ∅ then
12 |  |  B.add(v)
13 |  |  if alt is true    #Π is an odd integer then
14 |  |  |  v' ← the vertex from N s.t. the edge e(v,v') has the maximum #Ψ_e
15 |  |  else
16 |  |  |  v' ← the vertex from N s.t. the edge e(v,v') has the minimum #Ψ_e
17 |  |  e ← the edge linking v to v'
18 |  |  Π.add(e)
19 |  |  v ← v'
20 |  else
21 |  |  B ← B\{v}
22 |  |  ℵ ← the set of vertices v' s.t. ∃e(v,v') ∈ E\Π with v ∈ B
23 |  |  if alt is true    #Π is an odd integer then
24 |  |  |  e ← the edge e(v,v') s.t. v ∈ B  v' ∈ ℵ  e(v,v') that has the maximum #Ψ_e
25 |  |  else
26 |  |  |  e ← the edge e(v,v') s.t. v ∈ B  v' ∈ ℵ  e(v,v') that has the minimum #Ψ_e
27 |  |  Π.add(e)
28 |  |  v ← the endpoint vertex from e that is not in B
29 return Π
```

The concept of a connected edges ordering has the following intuition: The second edge in the sequence, e_2, must be adjacent to the first edge, e_1. Similarly, the third edge, e_3, must be adjacent to at least one of the preceding edges, and so forth. The connected edges order generated by Algorithm 3 is greedy because the initial edge, e_1, is selected based on the minimum number of promising node pairs (line 2 of Algorithm 3). The subsequent edge is chosen from among the adjacent edges of the previously selected edge (line 10 of Algorithm 3), with a preference for the edge with either the minimum or maximum number of promising node pairs (lines 11–19 of Algorithm 3). Since the subroutine `Get_Greedy_Connected_Edges_Order` is invoked with the parameter *alt* set to false, the strategy does not alternate between

minimum and maximum values. As a result, the next edge is always the adjacent edge with the fewest promising node pairs. The alternating greedy strategy (with *alt* set to true) is used during the joining phase in a later step of the algorithm. If no adjacent edges are available during the generation of the edge order, the next edge is chosen from those connected to any of the previously added edges (lines 20–28 of Algorithm 3). The variable *B* (line 4 of Algorithm 3) tracks vertices from already added edges that still have remaining adjacent edges. This set aids in selecting the next edge when no adjacent edges are available to the most recently added edge.

At this stage, the algorithm identifies a subset of edges, referred to as skip-edges, for which computing candidate pairs of objects is unnecessary (line 8 of Algorithm 3). This exception applies to inclusive edges that share endpoints with other edges for which candidate pairs have already been computed. Definition 4.9 formally defines the concept of a skip-edge.

Definition 4.9. Skip-edge (adapted from Fang et al. 2018a; Chen *et al.* 2020; Minervino et al. 2023; Pontes et al. 2025). *Given an ordered sequence of edges* $\Pi = (e_1, e_2, \ldots, e_m)$ *from a spatial pattern graph G, an edge* e_k *in* Π *is classified as a skip-edge if* e_k *is an inclusive edge and both of its endpoint vertices* v_i *and* v_j *are present in edges of the sub-sequence* $(e_1, e_2, \ldots, e_{k-1})$ *that precede* e_k.

For skip-edges, promising object pairs are inferred by filtering candidate solutions from adjacent edges, retaining only those that meet the skip-edge constraints. For non-skip edges, QQESPM-Quadtree computes candidate object pairs by assessing the distances and connectivity of all object pairs within the promising node pairs at the leaf level (Lines 9–10 of Algorithm 12). For topological relationships other than 'disjoint', it is more efficient to test bounding box intersections rather than exact connectivity checks, as the latter is computationally intensive. This minimizes unnecessary topological computations for objects that will likely be excluded, thereby enhancing efficiency. The concept of skip-edges, which avoids computing candidate object pairs by exhaustive objects verification within the promising nodes, is also present in classical algorithms for the simpler SPM query, such as MSJ (Fang et al. 2018a) and ESPM (Chen *et al.* 2020).

When computing promising node and object pairs for the edges, the algorithm often needs to verify whether a pair of nodes or objects *A* and *B* satisfies the constraints for an exclusive edge. To check the exclusion constraint, it performs a range query centered on *A* to determine if any object too close to *A* shares the same keyword as *B*. In such cases, QQESPM-Quadtree uses a memoization technique. The algorithm stores results of these range queries in temporary memory, thus avoiding redundant tests throughout a QQ-SPM query. This pruning based on exclusion constraints may occur during either the nodes' filtering phase (Line 14 of Algorithm 4.3.2) or the objects' filtering phase (Line 10 of Algorithm 1).

Consider an edge *e* with endpoints labeled 'apartment' and 'school', where $l_{ij} = 100$ m, $u_{ij} = 1000$ m, and the exclusion sign $\tau =$ '$\rightarrow$'. To determine if a pair of objects (A_1, S_1) qualifies as a matching pair for edge *e*, the exclusion constraint must be checked. Specifically, it needs to be verified whether a school *S* exists within a distance less than l_{ij} from A_1. This involves performing a radius search centered on A_1

with radius $r = l_{ij} = 100$ m to locate objects with the keyword 'school' in this area. If such an object is found within this radius, the pair (A_1, S_1) is disqualified as a matching pair for edge e. Furthermore, through computation reuse, A_1 is excluded from being paired with any other schools. The results of the radius search are stored in memory throughout the query, preventing the need to test identical radius searches centered on A_1 repeatedly.

After computing candidate pairs of objects for each non-skip edge, QQESPM-Quadtree reorders these edges for the joining phase (Line 11 of Algorithm 1). This reordering uses the function `Get_Greedy_Connected_Edges_Order` (Algorithm 3), which was previously invoked in Line 7. At this stage, the *alt* parameter for this subroutine is set to *true*, indicating that the greedy edge ordering will alternate between those with the minimum and those with the maximum number of promising object pairs. This connected, greedy alternating strategy helps avoid costly nested loops in joining candidate object pairs across all edges.

We assert that the order of edges during the final join phase significantly affects query performance and computational cost. Section 4.3.3 provides an example illustrating how different edge orders for processing impact the computational cost of the join phase of the query resolution. The connected greedy alternating strategy employed by QQESPM-Quadtree distinguishes it from ESPM and QQESPM algorithms (although ESPM only solves distance-based SPM queries). These existing algorithms reorder edges based solely on the number of candidate object pairs in ascending order. This strategy can be very inefficient, potentially leading to a disconnected edges orders that accumulates large numbers of irrelevant candidate solutions during the join phase, resulting in expensive nested loops and late discarding of unpromising solutions. In contrast, QQESPM-Quadtree ensures a connected edges order for the join phase, thereby preventing the rapid accumulation of large numbers of partially-constructed solutions.

After selecting the edges' order, QQESPM-Quadtree moves to the joining routine, which combines promising object pairs for adjacent edges and removes mismatches (Lines 12–14 of Algorithm 1). Specifically, edges sharing a common vertex must match the same geo-textual object at that vertex to form a valid join. Once the candidate object pairs for all non-skip edges are joined, the algorithm evaluates the resulting object groups and discards those that fail to meet the constraints of the skip-edges (Lines 15–16 of Algorithm 1). Finally, it verifies the actual topological relationships among the objects within the final candidate solutions to determine the ultimate solutions (Lines 17–18 of Algorithm 1). This final verification is necessary because, as noted in Line 10, all topological constraints except 'disjoint' were only roughly assessed through bounding box intersection checks. The algorithm then returns the set ψ of all object groups that match the spatial pattern G specified in the query.

Theorem 4.10. Adapted from Chen *et al.* (2020). *Let $G(V, E)$ be a spatial pattern to be queried against a dataset D of geo-textual objects. The time complexity of the QQESPM-Quadtree algorithm is $O(|D_w|^n)$, where $n = |V|$, and $|D_w| = \max\{|D_i||D_i$ is the set of objects with the keyword of vertex $v_i\}$.*

Proof. Computing promising node pairs and matching pairs of objects for all edges takes $O(m|D_w|^2)$, where $m = |E|$. For joining the object pairs across the edges to compose the final complete solutions of the spatial pattern, the operation is $O(|D_w|^n)$, such that the sum is also $O(|D_w|^n)$. ■

Notably, the worst-case time complexity of the QQESPM-Quadtree is equivalent to that of the ESPM algorithm (Chen *et al.* 2020) and its proof is analogous. The inclusion of topological relation checks does not affect the complexity, as containment and intersection operations between polygons remain linear with respect to the total number of polygon vertices, which are typically few in practical geo-textual applications such as POIs. Experimental results in Section 5 demonstrate that the QQESPM-Quadtree achieves faster average query execution times than ESPM. This improvement is attributed to the QQESPM-Quadtree's greedy, alternating join strategy, which contrasts with ESPM's simple unordered join approach. The space complexity of QQESPM-Quadtree is also given by the memory allocation of partial solutions during joining phase, which is also $O(|D_w|^n)$ since there is no garantee for each joining of a pair of edges that some of the combinations will be eliminated. Notably, the worst case complexity of QQESPM-Quadtree is analogous to the exhaustive search through the objects, although in average scenarios the quadtrees filter out most of the object pairs from testing since their nodes are not sufficiently close to satisfy distance requirements, and after promising node pairs computation, only reasonably distant nodes have their inner objects tested.

4.3.3. Choosing an order for joining edges

After computing the candidate pairs of objects for each edge, the algorithm must join these candidate solutions edge by edge to find the solutions for the entire search graph. The order in which edges are processed during this joining step can significantly affect the computational cost.

Consider a search graph with three edges ($e_{1,2}$, $e_{2,3}$, $e_{3,4}$), where each edge $e_{i,j}$ connects vertices v_i and v_j. Suppose edge $e_{1,2}$ has 1,000 candidate object pairs, edge $e_{2,3}$ has 10,000 candidate object pairs, and edge $e_{3,4}$ has 40 candidate object pairs. If the edges are joined in the order ($e_{1,2}$, $e_{2,3}$, $e_{3,4}$), the first step will join the solutions of $e_{1,2}$ and $e_{2,3}$, requiring $1,000 \times 10,000 = 10,000,000$ pair-wise tests, which may cause a significant overhead. If this join yields 20,000 solutions, these partial solutions must then be joined with the 40 solutions of edge $e_{3,4}$, requiring $20,000 \times 40 = 800,000$ additional computations. Thus, the total operations performed would be $10,000,000 + 800,000 = 10,800,000$.

Alternatively, if the edge solutions are joined in the order ($e_{3,4}$, $e_{2,3}$, $e_{1,2}$), the first step will join the 40 solutions of edge $e_{3,4}$ with the 10,000 solutions of edge $e_{2,3}$, requiring 400,000 operations. If this step yields 500 partial solutions, the next step will join these 500 partial solutions with the 1,000 solutions of edge $e_{1,2}$, costing 500,000 operations. For this strategy, the total operations performed would be $400,000 + 500,000 = 900,000$, which is significantly fewer than the 10,800,000 operations required by the previous strategy.

This example illustrates that the order of edges during the join phase can significantly affect the computational cost. In practice, predicting the exact number of operations for a specific edge order before the joining process is challenging, as the number of successfully joined pairs remains unknown until the joins are executed. To address this, the QQESPM-Quadtree algorithm employs the connected greedy alternating ordering strategy, as outlined in Algorithm 3. This strategy aims to achieve median performance in terms of operational cost by minimizing the magnitude of partially constructed solutions during the join phase. Additionally, a connected ordering filters candidate solutions at each join, while a non-connected ordering may lead to a full Cartesian product, resulting in a large number of candidates for subsequent joins.

4.4. QQESPM-eo

This section introduces another algorithm for addressing the QQ-SPM query, named QQESPM-EO. This algorithm effectively utilizes three fundamental geo-textual operations to comprehensively solve a QQ-SPM query. These operations are detailed as follows:

- EO_1 (**search by keyword**): Retrieves all objects containing a specified keyword.
- EO_2 (**search by distance and keyword**): Retrieves all objects that contain a specified keyword and are located within a specified distance from a given point.
- EO_3 (**search by topological relationship and keyword**): Retrieves all objects that contain a specified keyword and hold a particular topological relationship with the spatial boundary of a particular object.

The search procedure of QQESPM-Elastic is outlined in Algorithm 4 (QQESPM-EO), which utilizes the elementary operations EO_1, EO_2, and EO_3. The algorithm takes a dataset D of geo-textual objects and a spatial pattern graph $G(V, E)$ as inputs and produces ψ, representing all matches of G within D. The procedure begins by computing the frequency of each keyword within the search pattern (Lines 2 and 3 of Algorithm 4.4). If any keyword is missing from the dataset, the search pattern yields no solutions (Lines 4 and 5 of Algorithm 4). The algorithm then selects the initial vertex v with the lowest keyword frequency in the dataset to initiate the search (Line 6 of Algorithm 4). Using EO_1, it identifies all objects containing this keyword (Line 7). The variable Θ tracks the edges that have been processed, while B keeps a record of visited vertices with remaining edges.

Algorithm 4: QQESPM-EO

Input: D: dataset of geo-textual objects in a Elasticsearch index, $G(V,E)$: spatial pattern
Output: ψ: the matches of the spatial pattern G

1 $K \leftarrow$ // the set of keywords of all vertices in V
2 **for** *each keyword* $w \in K$ **do**
3 $\quad f_w \leftarrow$ the frequency of keyword w in the dataset D
4 **if** $\exists w \in K : f_w = 0$ **then**
5 $\quad$ **return** $\emptyset$
6 $v \leftarrow$ the vertex $v \in V$ that has the minimum $f_{v.keyword}$
7 $C_v \leftarrow$ RUN EO_1 for *v.keyword* in dataset D // the candidate objects for the vertex v
8 $\Theta \leftarrow \emptyset$ // processed edges
9 $B \leftarrow \{v\}$ //keep visited vertices with remaining edges
10 **while** $E \backslash \Theta \neq \emptyset$ *(there are edges left)* **do**
11 $\quad N \leftarrow \{v' \in v.\text{neighbors: } e(v,v') \notin \Theta\}$
12 $\quad$ **if** $N \neq \emptyset$ **then**
13 $\quad\quad B$.add(v)
14 $\quad\quad v' \leftarrow$ the vertex $v_i \in N$ that minimizes $f_{v_i.keyword}$
15 $\quad\quad e \leftarrow$ the edge linking v to v'
16 $\quad\quad S_e \leftarrow$ RUN Get_Object_Pairs_EO($e(v,v'), C_v$) // the promising object pairs for edge e
17 $\quad\quad \Theta$.add(e)
18 $\quad\quad$ Filter C_v and $C_{v'}$ to only the objects appearing at S_e
19 $\quad\quad$ Update the frequencies of keywords $f_{v.keyword}$ and $f_{v'.keyword}$ using S_e
20 $\quad\quad v \leftarrow v'$
21 $\quad$ **else**
22 $\quad\quad B \leftarrow B \backslash \{v\}$
23 $\quad\quad$ **if** $B \neq \emptyset$ **then**
24 $\quad\quad\quad v \leftarrow$ the vertex $v \in B$ that has the minimum $f_{v.keyword}$
25 $E \leftarrow$ RUN Get_Greedy_Connected_Edges_Order(S, $G(V,E)$, true)// reorder the edges for the join phase
26 $\psi \leftarrow \emptyset$ // keep the partially constructed matches of the search pattern
27 **for** *each edge* $e \in E$ **do**
28 $\quad \psi \leftarrow \psi$.join($S_e$)
29 **return** ψ

The algorithm uses a `While` loop to traverse the edges of the search pattern as long as edges remain (Line 10 of Algorithm 4). It selects the next vertex from the neighbors of the current vertex. If there are neighboring vertices with remaining edges, the next vertex v' is chosen from these neighbors, as the vertex with the lowest keyword frequency in the dataset. The promising object pairs for the edge $e(v,v')$ are then computed using the `Get_Object_Pairs_EO` subroutine described in Algorithm 5. For each candidate object o_i corresponding to vertex v, the related objects o_j for vertex v' are identified as pairs (o_i, o_j) that meet the spatial constraints of edge e. If the edge is qualitative with a relationship different than 'disjoint', the algorithm employs the elementary operation EO_3 (filter by

topological relation with o_i). For quantitative edges, it uses operation EO_2 (filter by distance from o_i). This process is detailed in Lines 1–8 of Algorithm 5. The promising object pairs for edge e are returned to Algorithm 4 in the variable S_e (Line 16 of Algorithm 4).

Algorithm 5: Get_Object_Pairs_EO

Input: $e(v, v')$: an edge of a spatial pattern, C_v: the candidate objects for the vertex v
Output: S_e: the promising object pairs for edge e

1 $S_e \leftarrow \emptyset$
2 **for** $o_i \in C_v$ **do**
3 **if** *edge e is qualitative with relation* $\Re \neq$ *'disjoint'* **then**
4 $O_j \leftarrow$ RUN EO_3 for $(o_i, v'.keyword, \Re)$ in D // the set of objects o_j s.t. the pair (o_i, o_j) is promising for edge e
5 **else**
6 $O_j \leftarrow$ RUN EO_2 for $(o_i, v'.keyword, e.l_{ij}, e.u_{ij})$ in D // the set of objects o_j s.t. the pair (o_i, o_j) is promising for edge e
7 $S'_e \leftarrow$ the set of pairs (o_i, o_j) with $o_j \in O_j$
8 $S_e \leftarrow S_e \cup S'_e$
9 **return** S_e

The edge e is then marked as resolved (Line 17), and the candidate objects for the endpoint vertices of e are updated by filtering to include only those present in S_e. Additionally, the keyword frequencies associated with the vertices of e are updated to reflect the total counts of candidate objects for these endpoints. If the current vertex v has no neighboring vertices with remaining edges, the next vertex is selected from the visited vertices with remaining edges, stored in variable B, similarly choosing the one with the lowest keyword frequency (Lines 21–24 of Algorithm 4).

The edges are reordered to minimize the cost of the subsequent joining phase, which integrates the promising object pairs for all edges into solutions for the complete search pattern graph. This reordering follows the same strategy as in QQESPM-Quadtree, as detailed in Algorithm 3 (Line 25 of Algorithm 4). It uses a connected greedy alternating strategy to create an edge sequence that reduces computational cost during the joining step. The joining process mirrors that of QQESPM-Quadtree, eliminating promising object pairs that do not share a common object at the vertices of adjacent edges (Lines 26–28 of Algorithm 4). The final joined solutions are then returned in the variable ψ (Line 29).

The worst case time and space complexity of QQESPM-EO is analogous to QQESPM-Quadtree. However, the average performance of QQESPM-EO is strongly influenced by the backend spatial engine that is computing the elementary operations throughout the greedy path on the spatial pattern graph.

4.5. QQESPM-SQL

The QQESPM-SQL resolution method is a pipeline designed to solve QQ-SPM queries by first translating the spatial pattern requirements into a comprehensive SQL spatial

query that encompasses all constraints of the search pattern. This spatial SQL query can then be executed within a spatial relational database environment containing a dataset of geo-textual objects. The query then identifies the groups of objects matching the QQ-SPM query graph.

To construct the SQL query from the spatial pattern graph, two main strategies can be employed: an implicit join strategy or an explicit join strategy with an efficient greedy order. The pipeline for constructing the SQL query using the implicit JOIN strategy is outlined as follows.

4.5.1. QQESPM-SQL query construction pipeline with implicit JOIN strategy

1. Generate the WITH clause: create aliases for Common Table Expression (CTE) `tb_ < keyword >`, for each keyword in the search pattern *G*, containing all geo-textual objects (data rows) associated with the respective keyword;
2. Generate the SELECT clause: select the ID columns from all CTEs `tb_ < keyword >`, forming tuples that represent matches for the spatial pattern *G*;
3. Generate the FROM clause: list the CTE names `tb_ < keyword >` separated by commas, implementing an implicit JOIN strategy without specific strategic ordering;
4. Generate the WHERE clause: create logical conditions for the spatial constraints of all edges from *G*, connected by logical AND, in the order of edges aligned with the vertices order specified in the FROM clause.

An alternative method for constructing the SQL query from the spatial pattern involves using explicit INNER JOIN operations. We propose a greedy strategy to determine an optimal order for these JOINs. Specifically, the SQL query requires joining as many CTEs `tb_ < keyword >` as there are vertices in the search pattern. Consequently, instead of ordering edges, QQESPM-SQL must establish an order for vertices. Following the same rationale used for edge ordering in QQESPM-Quadtree and QQESPM-EO, the vertex order for arranging the CTEs `tb_ < keyword >` for the JOINs should ideally be a connected order. This ensures that each pairwise JOIN operation includes a filter with the spatial constraints of edges, thereby avoiding full Cartesian products during the JOINs. Definition 4.11 formally defines the conditions for a vertex order to be considered connected.

Definition 4.11. Connected Vertices Order. *An order of vertices* $\Pi = (v_1, v_2, \ldots, v_m)$ *from a spatial pattern graph G is termed a connected vertices order if, for each vertex* v_k *in* Π, v_k *is a neighbor (shares a common incident edge) with at least one vertex in the sub-sequence* $(v_1, v_2, \ldots, v_{k-1})$ *of its preceding vertices.*

Similar to QQESPM-Quadtree and QQESPM-EO, the explicit JOIN pipeline strategy of QQESPM-SQL employs an algorithm to establish a connected and greedy vertex order. This strategy mitigates heavy computations during the JOIN phase by managing the extent of partially constructed solutions at each JOIN step and preventing the accumulation of many irrelevant candidate solutions that would later be discarded. The algorithm determines the vertex order and consequently the sequence of CTEs `tb_ < keyword >` to be joined. This generation of this vertices' reordering is detailed in the routine `Get_Greedy_Connected_Vertices_Order` (Algorithm 6).

Algorithm 6. Get_Greedy_Connected_Vertices_Order

Input: F: the frequencies in dataset of the keywords of all vertices, $G(V, E)$: spatial pattern, *alt*: a boolean parameter (false specifies a connected-greedy-minimum reordering, true specifies a connected-greedy-alternating reordering

Output: Π: the vertices from V in a greedy and connected order

```
1  Π ← ∅ // keep the vertices already added to the reordering
2  v ← the vertex from V that minimizes F_{v.keyword}
3  Π.add(v)
4  B ← ∅ //keep visited vertices with remaining edges
5  while V\Π ≠ ∅ (there are vertices left) do
6      N ← {v' ∈ v.neighbors: v'∉Π}
7      if N ≠ ∅ then
8          B.add(v)
9          if alt is true    #Π is an odd integer then
10             v' ← the vertex v' ∈ N s.t. v' has the maximum F_{v'.keyword}
11         else
12             v' ← the vertex v' ∈ N s.t. v' has the minimum F_{v'.keyword}
13         Π.add(v')
14         v ← v'
15     else
16         B ← B\{v}
17         if B ≠ ∅ then
18             v ← the vertex v ∈ B that has the minimum F_{v.keyword}
19 return Π
```

Algorithm 6 takes as input the variable F, which contains the frequencies (number of objects in the dataset) for each keyword in the spatial pattern graph used in the query. Additionally, the spatial pattern graph G and the boolean parameter *alt* are provided, representing whether the vertices ordering for the query should follow an alternating greedy ordering strategy, similarly to Algorithm 3. For QQESPM-SQL, the query construction pipeline utilizes as default Algorithm 6 with *alt* set to true. This algorithm generates a connected, greedy, alternating vertex order, applying a similar greedy alternating strategy as in QQESPM-Quadtree and QQESPM-EO. The vertices of the search pattern are sequentially added to the array variable Π until none remain. The initial vertex is selected based on having the least frequent keyword in the dataset (Line 2 of Algorithm 6). The variable B tracks visited vertices with remaining edges, and is used when the current vertex lacks neighbors. In such cases, the next vertex must be a neighbor of at least one previously added vertex in Π.

Vertices are added to Π using a greedy strategy until all vertices are included (Line 5 of Algorithm 6). The next vertex is preferably chosen from among the neighbors of the most recently added vertex (Line 6). If there are unselected neighbors, the algorithm opts for vertices with either the least or most frequent keyword in the dataset, following a greedy alternating strategy (Lines 7–14). When no neighbors are available, the algorithm selects the next vertex from those stored in B (Lines 15–18). Once all vertices are processed, the final greedy connected vertex reordering Π is returned.

We now introduce a second pipeline for SQL query construction in QQESPM-SQL. This explicit JOIN SQL query construction pipeline utilizes the connected greedy vertex order determined by Algorithm 6 for the spatial pattern graph of the query. It constructs INNER JOINs with the CTEs `tb_ < keyword >` ordered according to the connected greedy vertex order. The pipeline consists of the following steps:

QQESPM-SQL query construction pipeline with explicit greedy connected JOIN strategy.

1. Generate the WITH clause: Alias CTEs `tb_ < keyword >` for each keyword in the search pattern *G*, containing all geo-textual objects (data rows) that include the respective keyword.
2. Generate the SELECT clause: Select the ID columns from all CTEs `tb_ < keyword >`, forming tuples that match the spatial pattern *G*.
3. Generate the FROM clause: List the CTE names `tb_ < keyword >` for all vertices in *G* in the order Π produced by Algorithm 6, connecting them with explicit INNER JOINs. The boolean condition for each INNER JOIN that links a CTE `tb_ < keyword >` for the keyword of a vertex *v* includes all spatial constraints for the edges connecting *v* to any other vertex whose CTE `tb_ < keyword >` has already been included in the joins.

Both the implicit and explicit JOIN strategies in QQESPM-SQL adhere to fundamental principles for creating efficient SQL spatial queries. For instance, in Boolean expressions (in the WHERE clause for implicit JOINs or within ON clauses for explicit JOINs), the choice of distance function is made systematically based on the scenario. When implemented in PostgreSQL with PostGIS spatial extensions, functions such as ST_DWithin and ST_Distance/ST_DistanceSphere are available. Documentation and empirical tests indicate that ST_DWithin generally outperforms ST_Distance when one spatial location is fixed and the other is variable. Conversely, ST_Distance is more efficient for filtering conditions that involve the distance between two variable geometries. For example, to find a house and a school within 1 km of each other, ST_Distance or ST_DistanceSphere should be used. For exclusion constraints in QQ-SPM queries, such as 'find houses at least 500 meters away from cemeteries' each candidate house in the JOINs must be tested in a radius search with ST_DWithin to ensure that no cemeteries are within 500 meters. In our implementation of QQESPM-SQL within PostGIS, the EXISTS operator applied to the output results of ST_DWithin is used to verify the presence of results, which cause the failure of exclusion constraints. These examples illustrate informed principles that enable the QQESPM-SQL pipeline to construct efficient SQL queries for QQ-SPM queries.

The worst-case time and space complexity of QQESPM-SQL cannot be precisely determined, as this approach simply generates the equivalent SQL query to retrieve matching spatial patterns. Its performance depends directly on the underlying database engine and the query execution plan adopted by the system.

5. Performance experiments

This section provides a performance comparison of various methods for solving QQ-SPM queries, using a POI search scenario as a case study. We detail the pipeline for each QQ-SPM query resolution method, describe the datasets utilized, and explain the methodology for search patterns. The section concludes with a presentation of the comparison results, emphasizing the strengths and weaknesses of each investigated resolution method.

5.1. Experimental methodology

This section outlines the experimental methodology employed in the performance evaluations. We describe the different methods used to solve QQ-SPM queries, the datasets selected for the queries, and the spatial patterns generated to define the queries' parameters.

5.1.1. QQ-SPM resolution methods

The experiments involved comparing seven distinct implementations for solving QQ-SPM queries. Each of them applies a specific algorithm or pipeline to address the QQ-SPM queries. These implementations are described as follows:

- **ESPM+TV**: This method involves a reproduced implementation of the ESPM algorithm from previous research (Chen *et al.* 2020) to address the distance constraints of the QQ-SPM query. It then filters the retrieved groups based on **T**opological **V**erification, retaining only the groups that satisfy both the quantitative and qualitative constraints of the search pattern. This baseline approach is the only pre-existing method employed in the experiments. It provides a straightforward way to retrieve results for a QQ-SPM query, but it is not optimized for handling distance and topological filtering simultaneously.
- **QQESPM-Quadtree CGA**: This method involves applying the proposed QQESPM-Quadtree algorithm (Algorithm 1), which uses a **C**onnected **G**reedy **A**lternating order for the edges in the final join phase of the search procedure.
- **QQESPM-Quadtree CGM**: This method also employs the QQESPM-Quadtree algorithm but utilizes a **C**onnected **G**reedy **M**inimum ordering strategy for the edges. This can be achieved by setting the parameter *alt* to *false* in line 11 of Algorithm 1 within the subroutine Get_Greedy_Connected_Edges_Order.
- **QQESPM-Quadtree NoC**: This method applies the QQESPM-Quadtree algorithm with a **No**t necessarily **C**onnected edges ordering. This is done by replacing the connected edges reordering in line 11 of Algorithm 1 with a simple reordering of the edges, starting from the one with the fewest candidate object pairs to the one with the most pairs. This implementation is equivalent to the QQESPM algorithm proposed in previous research (Minervino et al. 2023; Pontes et al. 2025) and uses the same join order as the ESPM algorithm from Chen *et al.* (2020).
- **QQESPM-Elastic**: This method consists of implementing the QQESPM-EO algorithm (Algorithm 4) with Elasticsearch serving as the backend for elementary geo-textual retrieval operations.

- **QQESPM-SQL-Imp**: This method uses the QQESPM-SQL pipeline designed to convert the geo-textual constraints of the QQ-SPM query into a spatial SQL query with an **Imp**licit join strategy, which is then executed against a PostgreSQL database with the PostGIS spatial extension.
- **QQESPM-SQL-Exp**: This method employs the QQESPM-SQL pipeline designed to convert the geo-textual constraints of the QQ-SPM query into a spatial SQL query with an **Exp**licit join strategy, following the connected greedy alternating order for the vertices. The resulting SQL query is then executed against a PostgreSQL database with the PostGIS spatial extension.

These seven distinct methods for resolving QQ-SPM queries were implemented in Python and subsequently compared based on performance, specifically measuring query execution time in relation to the size of the data source for the queries. These implementations are equivalent in the sence that they all retrieve the same query results. To expedite query processing, the datasets of geo-textual objects used in the experiments were equipped with pre-computed centroids for all polygonal geometries. The variation methods of QQESPM-Quadtree (CGA, CGM, NoC) and the baseline ESPM+TV implementations adopted a from-scratch implementation of IL-Quadtree indexing on disk using HDF5, setting trees' maximum depth to 6 and leaf node maximum capacity as 500 objects.

PostgreSQL and Elasticsearch instances were installed locally on the machine. The QQESPM-SQL method utilizes the PostgreSQL database, leveraging spatial functions (distance and topological functions) from the PostGIS extension. The columns containing the POIs' polygonal geometries and centroids were indexed with spatial GiST and SPGiST indexes in PostgreSQL. The PostgreSQL configuration file *postgresql.conf* was optimized using the PGTune[1] tool. QQESPM-Elastic uses Elasticsearch as a backend for elementary operations. Elasticsearch natively employs geohash spatial indexing. The elementary operation EO_1 of QQESPM-EO uses the *match* filter from Elasticsearch, while EO_2 applies the *geo_distance* filter and the EO_3 operation uses the *geo_shape* filter. The FOR loop from lines 2 to 8 of Algorithm 5 is executed in parallel using Elasticsearch's Multisearch API, as the elementary operations for each FOR loop iteration are independent of one another. The ad hoc implementations, which are independent of PostgreSQL and Elasticsearch subsystems (including the three variations of QQESPM-Quadtree and the baseline ESPM+TV method) employed the *Shapely* Python library for topological computations.

5.1.2. Datasets

We used two datasets containing POI data within bounding boxes centered around Greater London, UK, collected from OpenStreetMap[2]. The statistics for these datasets are presented in Table 2. Dataset D1 is a smaller subset of Dataset D2. Table 3 provides a sample of data from the larger dataset (Dataset D2).

During a search, the QQ-SPM resolution methods scan the geo-textual objects in the dataset, searching for keywords from the search pattern. These keywords are expected to be found in specific columns (e.g., 'amenity', 'shop', 'tourism', etc.). For example, if a query includes the keyword 'cafe', the POI with id 249217389 in Table 3 (line 5) would be a candidate matching object.

Table 2. Datasets statistics.

Dataset	#POIs	Extension	Total keywords	Distinct keywords
D1	38,000	5.5 km × 5.5 km	39,378	514
D2	1,046,068	36km × 36 km	1,075,177	1,197

Table 3. Sample data from the datasets.

id	Name	Amenity	Shop	Tourism	Landuse	Leisure	Building	Geometry
991607788	Kensington West						Apartments	POLYGON ((−0.2131 51.4951, …
154951930							residential	POLYGON ((−0.1817 51.5246, …
669557409							residential	POLYGON ((−0.1474 51.5215, …
249217389	Moscos Cafe	cafe						POLYGON ((−0.1539 51.5131, …
149065743	No 74 Hair Beauty		hairdresser					POLYGON ((−0.1027 51.5243, …

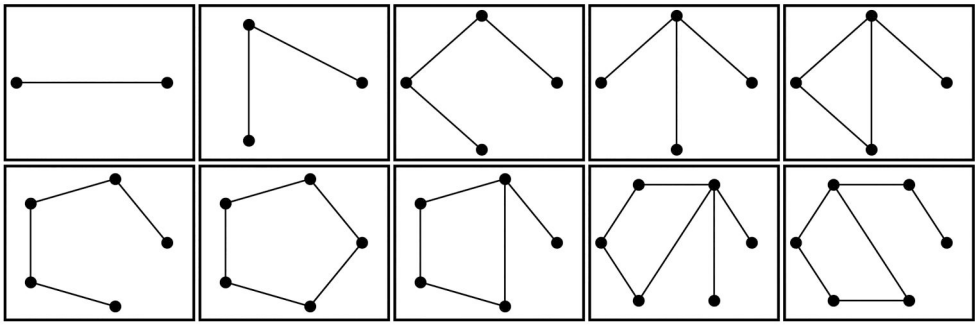

Figure 5. Architectures for the graphs of the spatial patterns for the queries.

5.1.3. Experiments description

Each dataset was used in separate batches of QQ-SPM query executions, which formed Experiments 1 and 2. The objectives of these experiments are outlined as follows:

- **Experiment 1**: Assess the performance of all investigated QQ-SPM resolution methods by running queries on a smaller dataset (Dataset D1).
- **Experiment 2**: Apply the most promising resolution methods, identified from Experiment 1, to a more extensive performance comparison using a larger dataset (Dataset D2) and more complex queries.

Spatial Patterns for Queries: The search patterns for Experiment 1 consisted of 144 randomly generated spatial patterns using the most frequent keywords from Dataset D1. For Experiment 2, another set of 144 search patterns was generated with the most frequent keywords from Dataset D2. Keywords included in the search patterns were those with a frequency greater than 30 in the respective datasets. Based on this threshold, a total of 123 frequent keywords were used to create the search patterns for Experiment 1, while 284 keywords were selected for Experiment 2. The

keywords for the query patterns were randomly chosen from the most frequent terms in the POI dataset, guided by intuition and the aim of designing search patterns likely to yield a high number of matches. This increases the computational cost of the queries, providing a more robust evaluation of the algorithms' performance.

Ten distinct graph architectures were utilized to generate the spatial patterns, as illustrated in Figure 5. An equal number of spatial patterns was generated for each graph architecture, except for the first two architectures, which received a doubled number of spatial patterns. Accordingly, for both Experiments 1 and 2, 24 spatial patterns were generated for each of the first two graph architectures, and 12 spatial patterns for each of the remaining architectures. This selection ensured a balanced number of queries for each possible number of vertices and edges in the search pattern.

Six different qualitative probabilities were considered to compose the search patterns in various configurations. The qualitative probability used in the generation of a spatial pattern represents the likelihood of each of its edges receiving a qualitative (topological) constraint. Equal numbers of spatial patterns were generated for each qualitative probability: 0, 0.2, 0.4, 0.6, 0.8, and 1. Spatial patterns generated with a qualitative probability of 1 are entirely based on topological constraints, while those with a qualitative probability of 0 rely solely on distance constraints.

The generated search patterns also incorporated varying distance, topological, and exclusion constraints for the edges. To define the distance constraints, the minimum distances l_{ij} were randomly selected between 0 and 1000 meters using a uniform distribution. Additionally, the maximum distance constraints were set between $l_{ij} + 200$ and $l_{ij} + 2000$ meters, ensuring a maximum search radius of 3 km between the queried POIs. These values were chosen to represent typical POI search scenarios within a single city, where the user seeks to find nearby POIs, ideally within the same neighborhood or district. For qualitative edges, the topological constraints were randomly and uniformly selected from 'contains', 'within', 'intersects', and 'disjoint'. The exclusion signs of the edges were randomly and uniformly selected from '$\leftarrow$', '$\rightarrow$', '$\leftrightarrow$', and '–'.

Query Executions: QQ-SPM queries were executed for all investigated implementations, including the baseline ESPM+TV. To evaluate the effect of dataset size on query execution time for the different resolution methods, smaller subsets were extracted from Dataset D1. The analysis examines how each method performs as the dataset size increases. Five subsets of varying sizes were created, representing different percentages of the total POIs: 20%, 40%, 60%, 80%, and 100%. The largest subset includes the entire dataset, randomly reordered. These subsets are used to assess the effect of dataset size on each resolution method as the dataset size grows. Similarly, this same subdivision procedure was applied to Dataset D2 to evaluate the influence of dataset size on the most promising resolution methods to be assessed by Experiment 2.

Experiment 1 involved 5,040 query executions to compare the performance of all seven methods across the five subsets of Dataset D1. In Experiment 2, a total of 2,160 query executions were conducted to compare the most promising methods from Experiment 1.

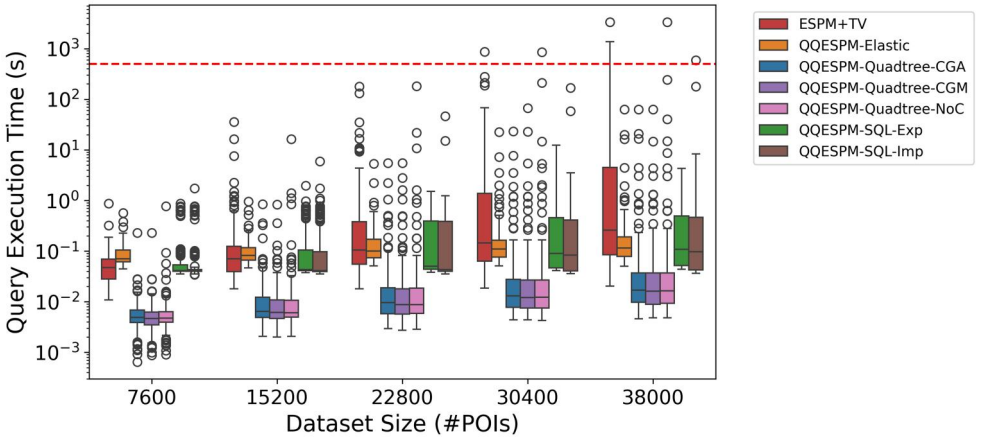

Figure 6. Statistics of Query Execution Time for the investigated resolution methods on increasing subsets of Dataset D1.

Environmental setting: The query executions were performed on an Ubuntu OS machine equipped with an Intel Core i7-12700F CPU running at 4.90 GHz and 64 GB of RAM.

5.2. Results

This section presents the results for Experiments 1 and 2.

5.2.1. Results of Experiment 1

Figure 6 displays the boxplots of query execution time statistics for all seven resolution methods across datasets of increasing sizes. These datasets consist of the five subsets of Dataset D1 (Experiment 1). Following a common convention, the boxplots' lower whiskers are limited by $Q_1 - 1.5 * (Q_3 - Q_1)$, and the upper whiskers by $Q_3 + 1.5 * (Q_3 - Q_1)$ where Q_1 and Q_3 are the 1st and 3rd quartiles of query execution times, respectively. In this sense, very uncommon (outlier) query execution times above or below these extremes are directly shown as individual data points in the plots. The median and interquartile execution times for QQESPM-Quadtree CGA, QQESPM-Quadtree CGM, and QQESPM-Quadtree NoC were nearly identical, while other resolution methods exhibited higher median execution times. The baseline method, QQESPM+TV, showed the poorest performance, with nearly exponential increases in both the interquartile range and maximum execution times. These results indicate that the baseline approach, ESPM+TV, lacked robustness and minimal scalability for QQ-SPM queries as the dataset size increased.

Although the QQESPM-Quadtree NoC method exhibits favorable median query times, its maximum execution times are comparable to those of the worst-performing method, ESPM+TV, significantly impacting the average query time. This discrepancy arises from the distinct edge ordering strategies used during the final join phase. The other two QQESPM-Quadtree methods (CGA and CGM) consistently employ a connected edge ordering, ensuring that each edge to be joined shares at least one

common vertex with the previously processed edges. This strategy reduces the number of pairwise checks required for each join, as candidate objects for each vertex are filtered to a smaller subset at each step. In contrast, QQESPM-Quadtree NoC does not guarantee a connected edge ordering, leading to scenarios where an edge may be joined without sharing a common vertex with already processed edges. This can result in a substantial number of pairwise checks, significantly degrading query performance.

A visual comparison of execution time ranges between the QQESPM-SQL-Imp and QQESPM-SQL-Exp methods reveals that both methods exhibit similar median and interquartile query times. However, only the QQESPM-SQL-Exp method demonstrates robustness as the dataset size increases, avoiding the high query times associated with QQESPM-SQL-Imp, which made the implicit JOIN strategy incur significantly higher maximum and average query times. Like QQESPM-Quadtree NoC and ESPM+TV, the QQESPM-SQL-Imp method also experiences substantial increases in query execution times for some queries, displaying nearly exponential growth in both maximum and average query times as execution time increases linearly.

The significant differences in maximum query times between QQESPM-SQL-Imp and QQESPM-SQL-Exp indicate that the structure of spatial SQL queries, particularly the order of JOIN operations, can greatly impact query performance. The experiment demonstrated that SQL queries with implicit JOINs in a random order resulted in higher query times compared to those with explicit JOINs arranged in a connected greedy alternating order. This suggests that our explicit JOIN ordering strategy, based on a connected greedy alternating sequence of vertices, is advantageous for complex SQL queries. It outperforms queries with implicit JOINs randomly ordered in the FROM clause. This discrepancy likely arises because the database system may not always identify the optimal query plan due to time constraints. Consequently, poorly structured implicit JOIN queries may fail to achieve the best query plan, while strategically ordered explicit JOIN queries benefit from enhanced optimization by the SQL query planner for QQ-SPM queries.

Notably, only three methods (ESPM+TV, QQESPM-Quadtree NoC, and QQESPM-SQL-Imp) exceeded 500 seconds for some queries, rendering these implementations impractical for larger datasets due to their lack of robustness as dataset size increases linearly. It is important to note that, as shown in Figure 6, the log-scaled y-axis visualization implies that nearly linear trends in maximum query time correspond to exponential increases on a real scale for these methods when dataset size grows linearly. Consequently, these three methods were excluded from Experiment 2 because they would consistently incur excessively high query times, making the experiments unfeasible.

Additionally, QQESPM-Quadtree CGA and QQESPM-Quadtree CGM demonstrated nearly identical performance as the dataset size increased. Notably, for the dataset with 30,400 POIs (the fourth x-value in Figure 6), the QQESPM-Quadtree CGA method exhibited a marginally better query time compared to QQESPM-Quadtree CGM. Consequently, QQESPM-Quadtree CGA showed a slight advantage in both average and maximum query times over QQESPM-Quadtree CGM.

As a result, only three methods were selected for Experiment 2, which involves a more detailed performance evaluation of the most promising methods from

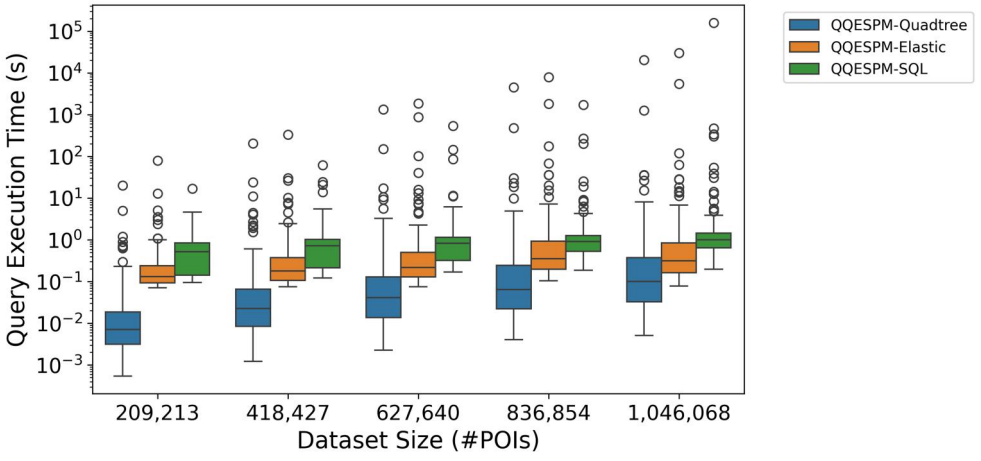

Figure 7. Statistics of Query Execution Time for the investigated implementations on increasing subsets of Dataset D2.

Experiment 1 based on query time robustness with larger datasets. Specifically, QQESPM-Quadtree CGA, QQESPM-Elastic, and QQESPM-SQL-Exp will be further assessed in Experiment 2. For simplicity, these methods will henceforth be referred to as QQESPM-Quadtree, QQESPM-Elastic, and QQESPM-SQL.

5.2.2. Results of experiment 2 for dataset size

Figure 7 presents the query execution times for the three most promising resolution methods (QQESPM-Quadtree, QQESPM-Elastic, and QQESPM-SQL) on subsets of Dataset D2 (Experiment 2). The boxplots, which summarize median and interquartile execution times, show that QQESPM-Quadtree consistently achieves the best performance across different dataset sizes, followed by QQESPM-Elastic and QQESPM-SQL. When considering maximum execution times, however, the results reveal a more nuanced scenario. For datasets with up to 837,000 Points of Interest (POIs), QQESPM-SQL exhibits slightly better robustness with respect to maximum query time. For full-dataset queries (approximately 1 million POIs), QQESPM-Quadtree delivers the lowest maximum execution time. Among the tested spatial search patterns, the maximum recorded query times were 20,599s, 29,971s, and 159,417s for QQESPM-Quadtree, QQESPM-Elastic, and QQESPM-SQL, respectively. Focusing on extreme but more realistic cases, the 99th-percentile execution times reached 35s, 164s, and 189s for QQESPM-Quadtree, QQESPM-Elastic, and QQESPM-SQL, respectively. Across all 720 spatial queries evaluated per method in Experiment 2, the mean execution times were 40s, 68s, and 228s for QQESPM-Quadtree, QQESPM-Elastic, and QQESPM-SQL, respectively. These results confirm the scalability and robustness of all three QQ-SPM query resolution strategies on large-scale datasets containing approximately 1 million spatial objects.

5.2.3. Results of experiment 2 for total query constraints

Figure 8 illustrates the distribution of query execution times versus the number of vertices in the spatial search patterns for the three evaluated implementations. The execution

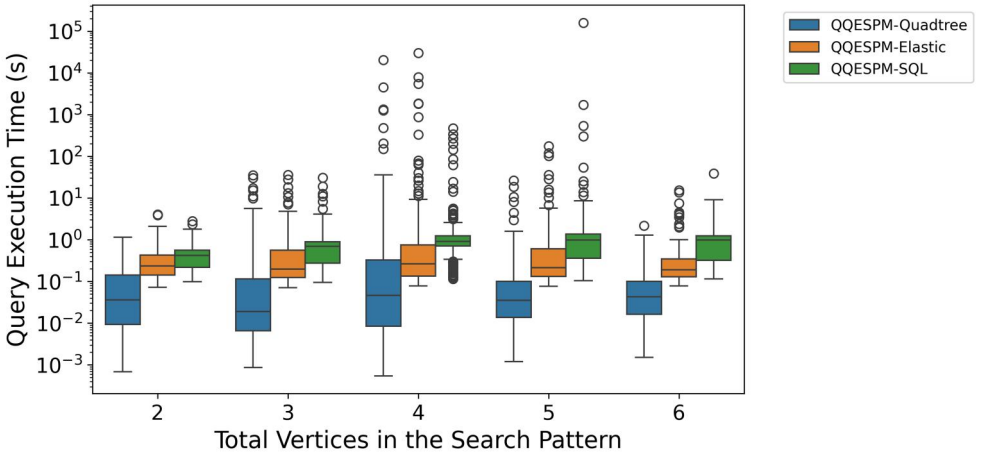

Figure 8. Statistics of Query Execution Time for the investigated implementations by total vertices in the search pattern for queries executed on Dataset D2.

times of the queries from Experiment 2 were grouped by patterns containing between 2 and 6 vertices. Across all vertex counts, QQESPM-Quadtree consistently achieved the best interquartile performance, followed by QQESPM-Elastic and, lastly, QQESPM-SQL. The results also show that queries involving 4 or 5 vertices produced notably high execution times in all three approaches. Execution time was strongly influenced by both the number of join operations and the volume of results returned. Patterns with more vertices introduce additional join operations due to the higher number of edges in the search graph, which increases processing cost. At the same time, such queries typically yield fewer results due to more restrictive spatial constraints. In contrast, patterns with fewer vertices require fewer joins and generally return larger result sets. As a consequence, queries with 4 to 5 vertices, or equivalently, 3 to 5 edges, tended to incur the highest execution costs, as they combined a relatively high number of joins with result sets that were still substantial. Overall, the analysis suggests that QQESPM-Quadtree exhibited slightly greater robustness across different pattern complexities.

Figure 9 presents the query execution times, for the three methods, grouped by the number of edges in the spatial query patterns. In a QQ-SPM query, each edge in the query's spatial pattern graph represents a spatial distance or topological constraint between two POIs. As expected, the number of edges is strongly correlated with the number of vertices in the queries patterns (see Figure 5). Consequently, the performance trends observed with respect to edge count closely parallel those seen for vertex count. In particular, query patterns containing 3 to 4 edges frequently resulted in the highest execution costs, as they required many join operations while also yielding a substantial number of results.

Figure 10 presents the query execution times for the three investigated implementations grouped by the qualitative probabilities of the search patterns. The boxplots on the left correspond to query patterns with lower qualitative probability and, therefore, fewer topological constraints, whereas those on the right represent query patterns with higher qualitative probability and a greater number of spatial constraints. Ignoring outliers, the performance ordering of QQESPM-Quadtree, QQESPM-Elastic, and QQESPM-SQL remains

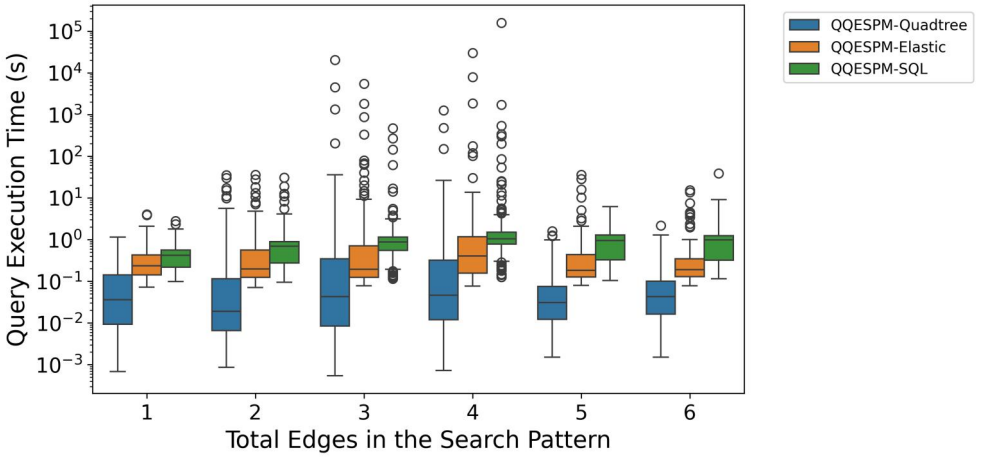

Figure 9. Statistics of Query Execution Time for the investigated implementations by total edges in the search pattern for queries executed on Dataset D2.

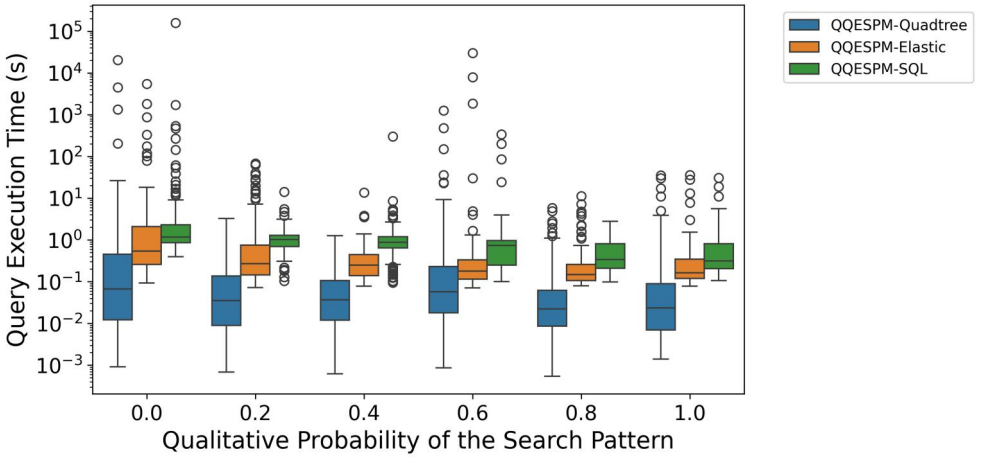

Figure 10. Statistics of Query Execution Time for the investigated implementations by Qualitative Probability of the Search Pattern for queries executed on Dataset D2.

consistent regardless of the qualitative probabilities of the queries. Overall, no strong correlation is observed between qualitative probability and execution time. However, queries with qualitative probability greater than or equal to 0.80 tend to exhibit more stable behavior, with fewer extreme latency outliers. These findings reinforce the effectiveness of the proposed implementations in handling search patterns with a large number of qualitative topological constraints, a key requirement for the QQ-SPM query model. QQESPM-Quadtree, in particular, demonstrates a clear robustness advantage due to its level-by-level node filtering strategy. By traversing the quadtree hierarchy and discarding unpromising node pairs early, based on the intuition that objects satisfying relations such as *contains*, *within*, and *intersects* must belong to the same or adjacent nodes, the method efficiently eliminates irrelevant combinations early. This significantly reduces

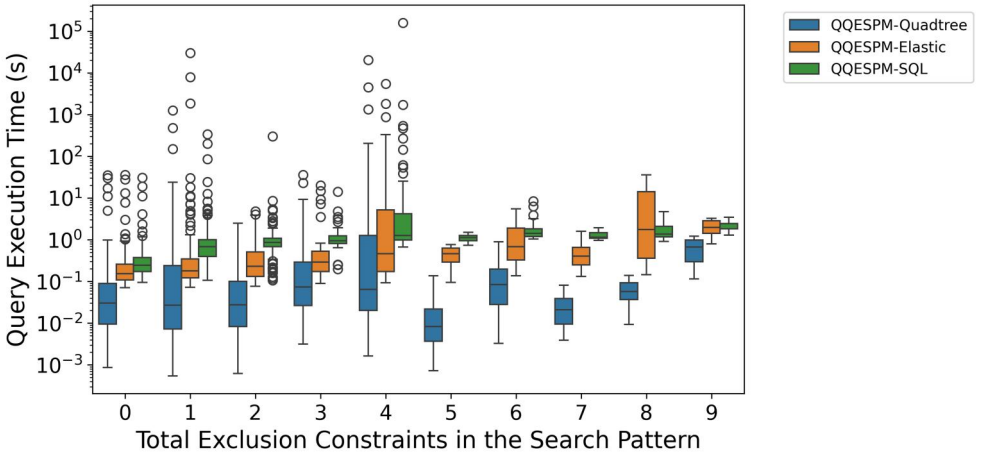

Figure 11. Statistics of Query Execution Time for the investigated implementations by total exclusion constraints in the search pattern for queries executed on Dataset D2.

the number of topological relation operations required, contributing to QQESPM-Quadtree's faster and scalable query resolution.

Figure 11 summarizes the query execution times grouped by the number of exclusion constraints of the search patterns. In the generated query patterns, the number of exclusion constraints ranged from 0 to 9. This number is computed as the sum of the exclusion constraints associated with each edge: edges marked '←' or '→' contribute one exclusion constraint, edges marked '↔' contribute two, and edges marked '–' contribute none. Ignoring outliers, the results suggest that QQESPM-SQL is strongly influenced by the number of exclusion constraints, as reflected in the generally increasing median and interquartile ranges of execution times for patterns with more exclusion constraints. In contrast, QQESPM-Quadtree and QQESPM-Elastic show more irregular variance, indicating that their performance is less sensitive to the number of exclusion constraints of the queries. Notably, QQESPM-Quadtree demonstrates a marked performance advantage for patterns with 5 or more exclusion constraints, primarily due to the memoization strategies employed in its implementation, which efficiently manage repeated evaluations of exclusion relationships during query execution.

6. Conclusions

This article explores QQ-SPM queries, a type of geo-textual search defined by a graph that represents spatial patterns with distance and topological constraints. These queries identify groups of geo-textual objects within a dataset that match the specified QQ-SPM spatial pattern. We illustrate the applicability of QQ-SPM queries in POI search scenarios, adhering to specific quantitative and qualitative spatial constraints. We propose two efficient algorithms for solving QQ-SPM queries: QQESPM-Quadtree and QQESPM-EO. Additionally, we introduce the QQESPM-SQL pipeline, which translates the geo-textual requirements of a QQ-SPM query graph into an efficient SQL query executable in spatial relational database systems. Collectively, these methods

form QQESPM-GS, a comprehensive approach for addressing spatial pattern queries across multiple environments for geo-textual data retrieval.

We systematically evaluate seven different implementations of resolution methods for QQ-SPM queries. These include three variations of the QQESPM-Quadtree algorithm, two of the QQESPM-SQL pipeline, the QQESPM-Elastic method (which implements QQESPM-EO using Elasticsearch backend), and a baseline approach based on the existing ESPM algorithm. Our experiments reveal that the baseline method is inefficient and highly unscalable. We find that the most effective QQESPM-Quadtree implementation uses a connected greedy alternating ordering for the final join phase. Similarly, the most efficient QQESPM-SQL implementation employs an explicit join strategy with a greedy ordering. In additional experiments with a larger dataset, QQESPM-Quadtree proved to be more robust, scalable and stable, consistently avoiding excessively high query times that other methods could not circumvent. In addition, it performed better for query patterns with large number of exclusion constraints. Open-source implementations of our proposed algorithms are available.

A limitation of the present study is that, although the QQESPM-Quadtree algorithm relies on initial parameter settings for the IL-Quadtree, such as node capacity and tree depth, and may also be influenced by the distribution of geo-textual objects in the dataset, the impact of varying these parameters was not investigated. Future research should therefore evaluate query performance under different parameter configurations to optimize the QQESPM-Quadtree's efficiency for datasets of varying sizes. Likewise, the influence of diverse data distributions on execution time warrants further examination.

Future research should focus on integrating these algorithms and pipelines natively into spatial databases and GIR systems. For instance, incorporating the proposed algorithms into the PostgreSQL database's native spatial environment could enhance the performance of SQL queries that follow the QQ-SPM query type. This integration would allow geotechnologies to automatically select optimized query plans for QQ-SPM queries. Additionally, adaptations for distributed computing environments should be considered to enable optimal processing of QQ-SPM queries in large datasets distributed across multiple clusters. A third avenue for future work is developing a pipeline that processes spatial queries in natural language, bridging QQ-SPM algorithms with natural language requirements.

Notes

1. https://pgtune.leopard.in.ua/
2. https://www.openstreetmap.org/

Disclosure statement

No potential conflict of interest was reported by the author(s).

Funding

This study was financed in part by the Conselho Nacional de Desenvolvimento Científico e Tecnológico – Brasil (CNPq).

Notes on contributors

Carlos Vinicius Alves Minervino Pontes holds an M.Sc. in Computer Science from the Federal University of Campina Grande (UFCG), Brazil, where he is currently affiliated as a researcher. His research interests include machine learning, spatial databases, spatial pattern matching, and spatio-textual query processing. In this paper, he contributed to the conceptualization of the approach, algorithm design, implementation, and experimental evaluation.

Dr. Claudio E. C. Campelo is an Associate Professor in the Systems and Computing Department at the Federal University of Campina Grande (UFCG), Brazil, and holds a Ph.D. in Computing from the University of Leeds (UK). His research interests include artificial intelligence, information retrieval, data modeling, and knowledge representation applied to geospatial information. In this paper, he contributed to the research design, methodological definition, and critical revision of the manuscript.

Dr. Maxwell Guimarães de Oliveira is an Adjunct Professor in the Systems and Computing Department at the Federal University of Campina Grande (UFCG), Brazil, and holds a Ph.D. in Computer Science from UFCG. His research interests include artificial intelligence, information retrieval, data modeling, and knowledge representation for geospatial applications. In this paper, he contributed to the research design, methodological definition, and critical revision of the manuscript.

Dr. Salatiel Dantas Silva is an Assistant Professor in the Systems and Computing Department at the Federal University of Campina Grande (UFCG), Brazil, and holds a Ph.D. in Computer Science from UFCG. His research focuses on spatial language, natural language processing, geoprocessing, artificial intelligence, and recommendation systems. In this paper, he contributed to the research design, methodological definition, and critical revision of the manuscript.

Data and codes availability statement

The data, codes, and instructions that support the findings of this study are available with the identifier(s) at: https://doi.org/10.6084/m9.figshare.28544636

References

Bedford, F.L., 1993. Perceptual and cognitive spatial learning. *Journal of Experimental Psychology. Human Perception and Performance*, 19 (3), 517–530.

Cao, X., *et al.*, 2015. Efficient processing of spatial group keyword queries. *ACM Transactions on Database Systems*, 40 (2), 1–48.

Cao, X., *et al.*, 2011. Collective spatial keyword querying. In *Proceedings of the 2011 ACM SIGMOD International Conference on Management of data*, p. 373–384.

Carniel, A.C., Ciferri, R.R., and Ciferri, C.D., 2020. FESTIval: A versatile framework for conducting experimental evaluations of spatial indices. *MethodsX*, 7, 100695.

Carniel, A.C., 2023. Defining and designing spatial queries: the role of spatial relationships. *GeoSpatial Information Science*, 0 (0), 1–25.

Cary, A., Wolfson, O., and Rishe, N., 2010. Efficient and scalable method for processing top-k spatial Boolean queries. *In*: M. Gertz and B. Ludäscher, eds. *Scientific and statistical database management* (Lecture Notes in Computer Science, Vol. 6187). Berlin, Heidelberg: Springer, 87–95. https://doi.org/10.1007/978-3-642-13818-8_8

Chan, H.K.H., Long, C., and Wong, R.C.W., 2017. Inherent-cost aware collective spatial keyword queries. In: *International Symposium on Spatial and Temporal Databases*, 357–375.

Chan, H.K.H., Long, C., and Wong, R.C.W., 2018. On generalizing collective spatial keyword queries. *IEEE Transactions on Knowledge and Data Engineering*, 30 (9), 1712–1726.

Chen, H., *et al.*, 2020. ESPM: Efficient spatial pattern matching. *IEEE Transactions on Knowledge and Data Engineering*, 32 (6), 1227–1233.
Chen, L., *et al.*, 2013. Spatial keyword query processing: An experimental evaluation. *Proceedings of the VLDB Endowment*, 6 (3), 217–228.
Chen, L., *et al.*, 2020. Spatial keyword search: A survey. *GeoInformatica*, 24 (1), 85–106.
Chen, Y., *et al.*, 2022. Example-based spatial pattern matching. *Proceedings of the VLDB Endowment*, 15 (11), 2572–2584.
Chen, Z., *et al.*, 2021. Location-and keyword-based querying of geo-textual data: a survey. *The VLDB Journal*, 30 (4), 603–640.
Choi, D.W., Pei, J., and Lin, X., 2016. Finding the minimum spatial keyword cover. In: *2016 IEEE 32nd International Conference on Data Engineering (ICDE)*, 685–696.
Choi, D.W., Pei, J., and Lin, X., 2020. On spatial keyword covering. *Knowledge and Information Systems*, 62 (7), 2577–2612.
Christoforaki, M., *et al.*, 2011. Text vs. space: efficient geo-search query processing. In: *Proceedings of the 20th ACM International Conference on Information and Knowledge Management*, 423–432.
Clementini, E., and Di Felice, P., 1995. A comparison of methods for representing topological relationships. *Information Sciences – Applications*, 3 (3), 149–178.
Clementini, E., Di Felice, P., and Van Oosterom, P., 1993. A small set of formal topological relationships suitable for end-user interaction. In: *International symposium on spatial databases*, 277–295.
Clementini, E., Sharma, J., and Egenhofer, M.J., 1994. Modelling topological spatial relations: Strategies for query processing. *Computers & Graphics*, 18 (6), 815–822.
Cohn, A.G., *et al.*, 1997. Qualitative spatial representation and reasoning with the region connection calculus. *GeoInformatica*, 1 (3), 275–316.
Cohn, A.G., and Renz, J., 2008. Qualitative spatial representation and reasoning. *Foundations of Artificial Intelligence*, 3, 551–596.
Cong, G., and Jensen, C.S., 2019. *Spatio-textual Data*. Cham: Springer International Publishing, 1580–1587.
Cong, G., Jensen, C.S., and Wu, D., 2009. Efficient retrieval of the top-k most relevant spatial web objects. *Proceedings of the VLDB Endowment*, 2 (1), 337–348.
de Almeida, J.P.D., and Rocha-Junior, J.B., 2015. Top-k spatial keyword preference query. *Journal of Information and Data Management*, 6 (3), 162–162.
De Felipe, I., Hristidis, V., and Rishe, N., 2008. Keyword Search on Spatial Databases. In: *2008 IEEE 24th International Conference on Data Engineering*, 656–665.
Deng, K., *et al.*, 2015. Best keyword cover search. *IEEE Transactions on Knowledge and Data Engineering*, 27 (1), 61–73.
Deng, K., *et al.*, 2017. Clue-based spatio-textual query. *Proceedings of the VLDB Endowment*, 10 (5), 529–540. https://doi.org/10.14778/3055540.3055546
Egenhofer, M., 1990. A mathematical framework for the definition of topological relations. In: *Proc. the fourth international symposium on spatial data handing*, 803–813.
Egenhofer, M. J., and Herring, J., 1990. *Categorizing binary topological relations between regions, lines and points in geographic databases. The 9-intersection: Formalism and its use for natural language spatial predicates*. Santa Barbara, CA: National Center for Geographic Information and Analysis. Technical Report. 94, 5–32. https://escholarship.org/uc/item/5nj6647c
Fang, Y., *et al.*, 2018a. On spatial pattern matching. In: *2018 IEEE 34th International Conference on Data Engineering (ICDE)*, 293–304.
Fang, Y., *et al.*, 2018b. SpaceKey: Exploring Patterns in Spatial Databases. In: *2018 IEEE 34th International Conference on Data Engineering (ICDE)*, 1577–1580.
Fang, Y., *et al.*, 2019. Evaluating pattern matching queries for spatial databases. *The VLDB Journal*, 28 (5), 649–673.
Fevgas, A., and Bozanis, P., 2019. LB-Grid: An SSD efficient grid file. *Data & Knowledge Engineering*, 121, 18–41.

Finkel, R.A., and Bentley, J.L., 1974. Quad trees a data structure for retrieval on composite keys. *Acta Informatica*, 4 (1), 1–9.

Fogliaroni, P., Weiser, P., and Hobel, H., 2016. Qualitative spatial configuration search. *Spatial Cognition & Computation*, 16 (4), 272–300.

Freksa, C., 1991. *Qualitative spatial reasoning. In*: D.M. Mark and A.U. Frank, eds. *Cognitive and linguistic aspects of geographic space.* Dordrecht, The Netherlands: Kluwer Academic Publishers, 361–372. https://doi.org/10.1007/978-94-011-2606-9

Gao, Y., Wang, Y., and Yi, S., 2016. Preference-Aware Top-k Spatio-Textual Queries. *In*: S. Song and Y. Tong, eds. *Web-age information management*. Cham: Springer International Publishing, 186–197.

Guo, T., Cao, X., and Cong, G., 2015. Efficient algorithms for answering the m-closest keywords query. In: *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data, SIGMOD '15*, Melbourne, Victoria, Australia New York, NY, USA: Association for Computing Machinery, 405–418.

Guo, X., and Yang, X., 2019. Direction-aware nearest neighbor query. *IEEE Access*, 7, 30285–30301.

Guo, Y., *et al.*, 2020. MCOPS-SPM: Multi-Constrained Optimized Path Selection Based Spatial Pattern Matching in Social Networks. In: *Cloud Computing, Smart Grid and Innovative Frontiers in Telecommunications: 9th EAI International Conference, CloudComp 2019, and 4th EAI International Conference, SmartGIFT 2019*, Beijing, China, December 4–5, 2019, and December 21–22, *9*, 3–19.

Guttman, A., 1984. R-trees: A dynamic index structure for spatial searching. In: Proceedings of *the 1984 ACM SIGMOD international conference on Management of data*, 47–57.

Hermoso, R., Ilarri, S., and Lado, R.T., 2019. *Re-CoSKQ: Towards POIs recommendation using collective spatial keyword queries*. In: *Proceedings of the RecTour Workshop at the ACM Conference on Recommender Systems (RecTour@RecSys 2019)*, Copenhagen, Denmark. CEUR Workshop Proceedings, Vol. 2435, 42–45. https://ceur-ws.org/Vol-2435/

Kalamatianos, G., Fakas, G.J., and Mamoulis, N., 2021. Proportionality in spatial keyword search. In: *Proceedings of the 2021 International Conference on Management of Data*, 885–897.

Khodaei, A., Shahabi, C., and Li, C., 2010. Hybrid indexing and seamless ranking of spatial and textual features of web documents. In: *Database and Expert Systems Applications: 21st International Conference, DEXA 2010, Bilbao, Spain, August 30-September 3, 2010, Proceedings*, *Part I 21*, 450–466.

Lee, T., *et al.*, 2015. Processing and optimizing main memory spatial-keyword queries. *Proceedings of the VLDB Endowment*, 9 (3), 132–143.

Li, G., Feng, J., and Xu, J., 2012. DESKS: Direction-Aware Spatial Keyword Search. In: *2012 IEEE 28th International Conference on Data Engineering*, 474–485.

Li, M., *et al.*, 2016a. Efficient processing of location-aware group preference queries. In: *Proceedings of the 25th ACM International on Conference on Information and Knowledge Management*, CIKM '16, Indianapolis, Indiana, USA New York, NY, USA: Association for Computing Machinery, p. 559–568.

Li, S., *et al.*, 2016b. Geospatial big data handling theory and methods: A review and research challenges. *ISPRS Journal of Photogrammetry and Remote Sensing*, 115, 119–133.

Li, Y., *et al.*, 2019. Spatial pattern matching: A new direction for finding spatial objects. *SIGSPATIAL Special*, 11 (1), 3–12.

Li, Z., *et al.*, 2011. IR-Tree: An efficient index for geographic document search. *IEEE Transactions on Knowledge and Data Engineering*, 23 (4), 585–599.

Long, C., *et al.*, 2013. Collective spatial keyword queries: a distance owner-driven approach. In: *Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data,* SIGMOD '13, New York, New York, USA New York, NY, USA: Association for Computing Machinery, p. 689–700.

Long, Z., *et al.*, 2016. Indexing large geographic datasets with compact qualitative representation. *International Journal of Geographical Information Science*, 30 (6), 1072–1094.

Lu, Y., *et al.*, 2013. SksOpen: Efficient Indexing, Querying, and Visualization of Geo-spatial Big Data. In: *2013 12th International Conference on Machine Learning and Applications*, Vol. 2, 495–500.

Mehta, P., *et al.*, 2016. Coverage and diversity aware top-k query for spatio-temporal posts. In: *Proceedings of the 24th ACM SIGSPATIAL International Conference on Advances in Geographic*

Information Systems, SIGSPACIAL '16, Burlingame, California New York, NY, USA: Association for Computing Machinery.

Minervino, C., *et al.*, 2023. QQESPM: A Quantitative and Qualitative Spatial Pattern Matching Algorithm. In: *XXIV Brazilian Symposium on Geoinformatics – GEOINFO 2023, INPE, São José dos Campos, SP, Brazil, December 4–6, 2023*, 49–60.

Moratz, R., and Ragni, M., 2008. Qualitative spatial reasoning about relative point position. *Journal of Visual Languages & Computing*, 19 (1), 75–98.

Pontes, C.V.A.M., *et al.*, 2025. QQ-SPM: spatial keyword search based on qualitative and quantitative spatial patterns. *Journal of Information and Data Management*, 16 (1), 82–91.

Qian, Z., *et al.*, 2018. Semantic-aware top-k spatial keyword queries. *World Wide Web*, 21 (3), 573–594.

Rafael, G.J.R., *et al.*, 2024. Topo-MSJ: Search for groups of POIs with qualitative processing of spatial regions. *Transactions in GIS*, 28 (7), 2195–2218.

Randell, D.A., Cui, Z., and Cohn, A.G., 1992. A spatial logic based on regions and connection. *KR*, 92, 165–176.

Rocha-Junior, J.B., *et al.*, 2011. Efficient processing of top-k spatial keyword queries. In: *Advances in Spatial and Temporal Databases: 12th International Symposium, SSTD 2011*, 24–26 August 2011 Minneapolis, MN, USA, *Proceedings* (Lecture Notes in Computer Science, Vol. 6849). Heidelberg, Germany: Springer, 205–222. https://doi.org/10.1007/978-3-642-22922-0_13

Roumelis, G., *et al.*, 2020. Parallel processing of spatial batch-queries using xBR+-trees in solid-state drives. *Cluster Computing*, 23 (3), 1555–1575.

Skovsgaard, A., and Jensen, C.S., 2015. Finding top-k relevant groups of spatial web objects. *The VLDB Journal*, 24 (4), 537–555.

Sun, J., *et al.*, 2017. Interactive spatial keyword querying with semantics. In: CIKM '17. Singapore, Singapore New York, NY, USA: Association for Computing Machinery, 1727–1736.

Tampakis, P., *et al.*, 2021. A novel indexing method for spatial-keyword range queries. *Proceedings of the 17th International Symposium on Spatial and Temporal Databases*, 54–63.

Tao, Y., and Sheng, C., 2014. Fast nearest neighbor search with keywords. *IEEE Transactions on Knowledge and Data Engineering*, 26 (4), 878–888.

Tsatsanifos, G., and Vlachou, A., 2015. *On processing top-k spatio-textual preference queries*. *EDBT*, 433–444.

Wu, D., Cong, G., and Jensen, C.S., 2012. A framework for efficient spatial web object retrieval. *The VLDB Journal*, 21 (6), 797–822.

Wu, D., *et al.*, 2012. Joint top-k spatial keyword query processing. *IEEE Transactions on Knowledge and Data Engineering*, 24 (10), 1889–1903.

Xu, T., *et al.*, 2022. Efficient processing of top-k frequent spatial keyword queries. *Scientific Reports*, 12 (1), 7352.

Yang, M., *et al.*, 2015. Cloud-assisted spatio-textual k nearest neighbor joins in sensor networks. *In: 2015 1st International Conference on Industrial Networks and Intelligent Systems (INISCom)*, 12–17.

Yiu, M.L., *et al.*, 2007. Top-k spatial preference queries. In: *2007 IEEE 23rd International Conference on Data Engineering*, 1076–1085.

Zhang, C., *et al.*, 2016. Inverted linear quadtree: Efficient top k spatial keyword search. *IEEE Transactions on Knowledge and Data Engineering*, 28 (7), 1706–1721.

Zhang, D., *et al.*, 2009. Keyword search in spatial databases: Towards searching by document. In: *2009 IEEE 25th international conference on data engineering*, 688–699.

Zhang, D., Ooi, B.C., and Tung, A.K.H., 2010. Locating mapped resources in Web 2.0. In: *2010 IEEE 26th International Conference on Data Engineering (ICDE 2010)*, 521–532.

Zhang, D., Tan, K.L., and Tung, A.K.H., 2013. Scalable top-k spatial keyword search. In: *EDBT '13*, Genoa, Italy New York, NY, USA: Association for Computing Machinery, 359–370.

Zhang, L., Sun, X., and Zhuge, H., 2014. Density-based spatial keyword querying. *Future Generation Computer Systems*, 32, 211–221.

Zhang, P., *et al.*, 2017. Level-aware collective spatial keyword queries. *Information Sciences*, 378, 194–214.